Online Supplementary Material: Probe-Guided Rebalancing of Persistent Subscription Connections

MANISH KUMAR AGRAWAL, Indian Institute of Technology Roorkee, India
SANDEEP KUMAR, Indian Institute of Technology Roorkee, India

CCS Concepts: • **Computing methodologies** → **Reinforcement learning**; • **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → *Message passing*; *Publish-subscribe / event-based architectures.*

Additional Key Words and Phrases: persistent connections, subscription rebalancing, deep Q-learning, partial observability, probe promotion, greedy control, messaging systems, autonomic control

This Online Supplementary Material accompanies the main manuscript *Probe-Guided Rebalancing of Persistent Subscription Connections with Greedy and Deep Q-Learning Control*. It retains the detailed mathematical, implementation, and experimental material relocated from the reference manuscript. Reported measurements, comparisons, and figures are unchanged. References explicitly marked "of the main paper" point to the separate main manuscript. Figures, tables, equations, and algorithms in this document use an S prefix.

Contents

Contents 1
A Extended Mathematical Details 2
B Notation Reference 4
C Detailed Runtime Algorithms and Checkpoint Persistence 8
D Evaluated Features and Policy Settings 18
E Extended Cloud and Edge Literature 19
F System Model 20
G Additional 20,000-Subscription Handoff Diagnostics 22
H Concentrated Placement and Startup Under Load 23
I TTL-Based Reconnection Under Delivery Load 26
J Additional Scale-Out and Timing-Control Results 33
K Full Supporting Simulator Studies 36
L Action-Contract Development Diagnostics 48
M Extended Interpretation 50
References 51

Authors' Contact Information: Manish Kumar Agrawal, m_agrawal@cs.iitr.ac.in, Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee, Uttarakhand, India; Sandeep Kumar, sandeep.garg@cs.iitr.ac.in, Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee, Uttarakhand, India.

A Extended Mathematical Details

The core queue, pressure, imbalance, and reward definitions appear in the main paper. This section retains their reference-scale calibration, the offline objective, and the full proofs.

A.1 Variance Reference Scales

The subscription-count reference V_{ref}^N is chosen from an operational tolerance rather than learned by DQL. Since $\sqrt{V_u^N(t)}$ is the standard deviation of subscription counts across client u's active connections, a tolerable subscription-count standard deviation σ_{tol}^N defines

$$V_{\text{ref}}^N = (\sigma_{\text{tol}}^N)^2.$$

For example, if the operator allows roughly a Δ_N-subscription spread between a heavy and a light connection, then $\sigma_{\text{tol}}^N \approx \Delta_N/2$ is a practical calibration. If the subscription-density threshold is Θ_N, a default choice is $\sigma_{\text{tol}}^N = \rho_N \Theta_N$ with $\rho_N \in [0.1, 0.25]$; smaller values increase sensitivity to subscription-count skew. The backlog-load reference V_{ref}^G is chosen in the same way, but in backlog units. Since $\sqrt{V_u^G(t)}$ is the standard deviation of cumulative connection backlog $G_{u,c}(t)$ across client u's active connections, a tolerable backlog-load standard deviation σ_{tol}^G defines

$$V_{\text{ref}}^G = (\sigma_{\text{tol}}^G)^2.$$

If the operator allows roughly a Δ_G-message backlog spread between a heavy and a light connection, then $\sigma_{\text{tol}}^G \approx \Delta_G/2$ is the direct calibration. When the tolerance is latency-driven, Δ_G is derived from the client-observed processing capability rather than from pod capacity alone. For example, if $\widehat{r}_u^{\text{ack}}$ is a conservative ACK or processing rate measured by client u and τ_G is the acceptable additional drain-time skew, then $\Delta_G = \widehat{r}_u^{\text{ack}} \tau_G$ is a defensible choice. If no latency SLO is available, a practical default is $\sigma_{\text{tol}}^G = \rho_G \beta \Theta_N$ with $\rho_G \in [0.1, 0.25]$, where $\beta\Theta_N$ is the nominal backlog budget of a connection carrying Θ_N subscriptions. The weights $\eta_N, \eta_G \geq 0$ determine how much the controller prioritizes subscription-management fairness versus backlog-load fairness. For example, larger η_N makes the controller more sensitive to connections carrying too many subscriptions, while larger η_G makes it more sensitive to connections accumulating more queued messages. The constraint $\eta_N + \eta_G = 1$ makes $V_u(t)$ a convex combination rather than an unbounded sum.

A.2 Global Evaluation Objective

The controller does not require cluster-wide variance. However, simulation and offline validation can report global metrics. A finite-horizon evaluation cost is

$$J_T = \sum_{t=0}^{T} \Big[\Lambda_B \sum_{i \in \mathcal{S}} B_i(t) + \Lambda_X \Phi_X(t) + \Lambda_C \sum_{u \in \mathcal{U}} V_u(t) + \Lambda_M M(t) \Big]. \tag{S1}$$

where $\Phi_X(t)$ is pod subscription-count variance and $M(t)$ is total migration count. This objective is used to evaluate system-level behavior, not as information assumed available to every client.

The quantity J_T is an offline finite-horizon cost: lower values indicate better system-level behavior over the evaluation horizon $0, \ldots, T$. The first term penalizes total pod backlog, where $B_i(t)$ is the cumulative backlog over all subscription

queues served by pod i. The second term penalizes uneven subscription placement across pods. Specifically, if

$$\bar{X}(t) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} X_i(t),$$
$$\Phi_X(t) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \big(X_i(t) - \bar{X}(t)\big)^2,$$

then $\Phi_X(t)$ is high when some pods carry many more subscriptions than others. The third term aggregates client-side connection imbalance, using $V_u(t)$ from (13 of the main paper); it measures whether a client's own persistent connections are uneven in subscription count or backlog load. The fourth term penalizes operational churn through $M(t)$, the number of migration or reconnect operations performed during epoch t. The coefficients $\Lambda_B, \Lambda_X, \Lambda_C, \Lambda_M \geq 0$ determine the relative importance of backlog reduction, pod-level subscription balance, client-side connection balance, and migration stability. Normalization before weighting places the components on comparable scales, preventing a large-scale metric from dominating J_T merely because of its units.

A.3 Rebalancing Properties

Proposition 1 (Subscription-density bound). *If* $1 \leq |\mathcal{K}_u(t)| \leq \Theta_N C_{\max}$ *and the scaling rule sets*

$$|C_u(t)| = \left\lceil \frac{|\mathcal{K}_u(t)|}{\Theta_N} \right\rceil, \tag{S2}$$

then the mean subscription density satisfies $\bar{N}_u(t) \leq \Theta_N$.

Proof. By definition, $\bar{N}_u(t) = |\mathcal{K}_u(t)|/|C_u(t)|$. Since $|C_u(t)| \geq |\mathcal{K}_u(t)|/\Theta_N$, it follows that $\bar{N}_u(t) \leq \Theta_N$. □

Proposition 2 (Backlog-variance reduction). *For client* u, *let* $c_h, c_l \in C_u(t)$ *be two active persistent connections, where* c_h *is the source connection from which subscription* k *is removed and* c_l *is the target connection to which subscription* k *is assigned. Let* $G_{u,c_h}(t)$ *and* $G_{u,c_l}(t)$ *be the cumulative backlog loads on these two connections before the move, and let* $b_{u,k}(t) > 0$ *be the backlog of the moved subscription queue. If*

$$G_{u,c_h}(t) - G_{u,c_l}(t) > b_{u,k}(t), \tag{S3}$$

then moving subscription k *from* c_h *to* c_l *strictly decreases the connection-level backlog variance* $V_u^G(t)$.

The condition states that the source connection is heavier than the target connection by more than the backlog carried by the moved subscription. Hence the move reduces their squared-deviation sum. It can cross the equal-load point and still reduce variance: loads $(10, 0)$ become $(2, 8)$ after a move of size 8. Avoiding that crossing would require the stronger condition $2b_{u,k}(t) \leq G_{u,c_h}(t) - G_{u,c_l}(t)$, which is not needed by the proposition.

Proof. The move preserves total backlog and therefore preserves $\bar{G}_u(t)$. Let $d_h = G_{u,c_h}(t) - \bar{G}_u(t)$ and $d_l = G_{u,c_l}(t) - \bar{G}_u(t)$. Only two squared-deviation terms change. With $b = b_{u,k}(t)$, the unnormalized variance change is

$$\begin{aligned}&(d_h - b)^2 + (d_l + b)^2 - d_h^2 - d_l^2 \\ &\quad = 2b\big(b - (G_{u,c_h}(t) - G_{u,c_l}(t))\big) < 0.\end{aligned} \tag{S4}$$

Thus $V_u^G(t)$ decreases. □

Remark 1. *The placement loop optimizes a client-observed reward rather than an unobserved global cluster objective. It is a decentralized control mechanism operating under partial observability: active connections and probe connections provide fresh but incomplete pod statistics, and the client optimizes its own reward using these observations.*

B Notation Reference

The tables collect the detailed definitions used in the model and experimental endpoints.

Table S1. Summary of Notations: System Sets and Pod Pressure

Symbol	Description
Server-Level Placement Variables	
$\mathcal{S} = \{1, \ldots, n\}$, $\mathcal{U} = \{1, \ldots, m\}$	Sets of Pod indices and client indices; n and m are their counts
t, T, Δ	Decision-epoch index, last epoch, and seconds per epoch; adjacent epochs are t and $t+1$
$\mathcal{K}_u(t)$, $C_u(t)$	Client u's subscription set and active delivery-connection set
$\mathcal{K}_{u,c}(t)$, $z_{u,k,c}(t)$	Subscriptions assigned to connection c and the binary indicator of that assignment
$Q_{u,k}(t)$, $b_{u,k}(t)$	Subscription queue and its backlog count, $b_{u,k}(t) = \lvert Q_{u,k}(t) \rvert$
$i_{u,c}(t)$, $O_u(t)$	Pod index reached by connection c and set of Pod indices observed by client u
p_i, $y_i(t)$	Pod name and reported tuple $(p_i, X_i(t), B_i(t), H_i)$; $y_i(t)$ is not the scalar Bellman target y_t
$\mathcal{R}_i(t)$	Set of server-side subscription queues currently served by pod i
$\tilde{b}_{i,r}(t)$	Backlog of server-side subscription queue $r \in \mathcal{R}_i(t)$
$X_i(t)$	Pod-level subscription count for pod i
$B_i(t)$	Cumulative backlog over all subscription queues served by pod i
H_i	Configured maximum processing threshold for Pod i in events/s, used as a normalization signal
X_{safe}	Subscription-count level after which pod reliability risk begins to increase
X_{hard}	Hard subscription-count reference used to normalize severe subscription pressure, with $X_{\text{hard}} > X_{\text{safe}}$
B_{ref}	Reference backlog used to normalize pod backlog pressure
$P_i^X(t)$	Normalized pod subscription-count pressure
$P_i^B(t)$	Normalized pod cumulative-backlog pressure
$P_i^H(t)$	Backlog pressure normalized by $H_i\Delta + \varepsilon_0$, then clipped at one
ε_0, $[x]_+$	Positive denominator offset in messages and positive part $\max\{x, 0\}$; distinct from exploration probability ϵ
$P_i^R(t)$	Nonlinear pod reliability-risk pressure caused by excessive subscription count
$P_i(t)$	Weighted pod pressure score used in the deployable reward
$\omega_X, \omega_B, \omega_H, \omega_R$	Nonnegative weights for subscription-count, backlog, threshold-relative backlog, and reliability-risk pressure terms

Table S2. Summary of Notations: DQL Observation and Reward Symbols

Symbol	Description
J_T	Offline finite-horizon system evaluation cost over epochs $0, \ldots, T$
$\Phi_X(t)$	Pod subscription-count variance across the cluster at time t
$M(t)$	Total number of migration or reconnect operations during epoch t
$\Lambda_B, \Lambda_X, \Lambda_C, \Lambda_M$	Nonnegative weights in the offline evaluation objective for backlog, pod balance, connection balance, and migration churn
$T_u(t)$	Effective client-side throughput from delivered, processed, and acknowledged messages
$R_u(t)$	Deployable reward observed by client u at time t
$Q_\theta(o, a)$	Online DQL action-value network used for action selection and gradient updates
$Q_{\theta^-}(o, a)$	Target DQL action-value network used only to compute Bellman targets
θ, θ^-	Online-network parameters and delayed target-network parameters
$\mathcal{D}, \xi_t$	Replay buffer and transition $(o_u(t), a_u(t), R_u(t), o_u(t+1))$
y_t, $\mathcal{L}(\theta)$	Scalar Bellman target and mean squared temporal-difference loss
α, K_{sync}	Online-network learning rate and target-copy interval in training updates
γ	DQL discount factor
ϵ	Exploration probability in the DQL epsilon-greedy action policy
δ	Required Q-value improvement of migration over stay; distinct from variance thresholds γ_N, γ_G
$\lambda_1, \ldots, \lambda_7$	Weight coefficients used in the deployable reward function
$o_u(t)$	Client observation vector used by the DQL policy
$o_u^B(t)$	Backlog observation branch containing normalized local backlog and backlog growth features
$o_u^N(t)$	Subscription-count observation branch containing normalized $N_{u,c}(t)$ and $V_u^N(t)$ features
$o_u^G(t)$	Backlog-load imbalance observation branch containing normalized $G_{u,c}(t)$ and $V_u^G(t)$ features
$o_u^C(t), o_u^P(t), o_u^D(t)$	Connection-count/throughput, pod-pressure, and decision-state observation branches
$\mathcal{A}$	Set of bounded DQL action templates
$\mathcal{A}_u^{\text{adm}}(t)$	Subset of action templates admitted by client u's guards at time t, used for action selection and next-state Bellman maximization
$i_u^{\text{probe}}(t)$	Pod endpoint bound to client u's currently retained, load-balancer-assigned probe socket
$a_u(t), a'$	Selected action at epoch t and candidate next-epoch action in Bellman maximization

Table S3. Summary of Notations: Connection Scaling and Local Counters

Symbol	Description
Connection Scaling Variables	
$C(t)$	Number of active persistent delivery connections for client u at time t, $C(t) = \lvert C_u(t)\rvert$
$C_{\max}$	Maximum allowable client connections
$N_{\text{sub}}(t)$	Total number of active subscriptions maintained by client u at time t, $N_{\text{sub}}(t) = \lvert\mathcal{K}_u(t)\rvert$
Θ_N	Maximum desired subscription count per connection used by the scaling rule
β	Backlog threshold per subscription
$b_{u,k}(t)$	Client-observed backlog of subscription queue k for client u
$C_{\text{sub}}(t)$	Required connections based on total subscription count
$C_k(t)$	Client connection-count requirement triggered by subscription k's backlog; not multiple owners for that queue
$C_{\text{back}}(t)$, $C^\star(t)$	Maximum backlog-based requirement and desired connection count after combining scaling requirements
c^{place}, $c^\star$	Connection chosen for new-subscription placement and connection evaluated for migration
$\widehat{N}_{u,c}(t)$, $\widehat{G}_{u,c}(t)$	Normalized subscription-count and backlog loads used by the placement score
$B_u(t)$, $\text{Grow}_u(t)$	Sum of client u's subscription backlogs and its positive increase since the previous epoch
$T_u^{\text{del}}(t)$, $T_u^{\text{proc}}(t)$, $T_u^{\text{ack}}(t)$	Delivered, processed, and acknowledged message rates in events/s; their minimum is $T_u(t)$
$\widehat{B}_u(t)$, $\widehat{\text{Grow}}_u(t)$, $\widehat{T}_u(t)$	Normalized backlog, backlog growth, and throughput under the experiment's stated scaling convention
$M_u(t)$, $\widehat{C}_u(t)$	Migration indicator or normalized count, and active connection fraction $C(t)/C_{\max}$

Table S4. Summary of Notations: Connection Rebalancing Symbols

Symbol	Description
Connection-Level Rebalancing Variables	
$N_{u,c}(t)$	Number of subscriptions of client u assigned to connection c at time t
$G_{u,c}(t)$	Cumulative backlog load on connection c, computed from the subscription queues assigned to that connection
c_h, c_l	Source high-load connection and target low-load connection used in a rebalancing move
$V_u^N(t)$	Connection-level subscription-count variance across client u's active persistent connections
$\bar{N}_u(t)$, $\bar{G}_u(t)$	Mean subscription count and mean cumulative backlog across client u's active connections
$V_u^G(t)$	Connection-level backlog-load variance across client u's active persistent connections

Continued on next page

Table S4. Summary of Notations: Connection Rebalancing Symbols (continued)

Symbol	Description
$\widehat{V}_u^N(t), \widehat{V}_u^G(t)$	Normalized subscription-count and backlog-load variance terms used in the local imbalance score
$V_{\text{ref}}^N, V_{\text{ref}}^G$	Positive variance references used to normalize $V_u^N(t)$ and $V_u^G(t)$
σ_{tol}^N	Tolerable standard deviation of subscription counts across a client's active connections
σ_{tol}^G	Tolerable standard deviation of cumulative backlog load across a client's active connections
Δ_N	Operator-chosen tolerable spread between a heavy and a light connection in subscription count
Δ_G	Operator-chosen tolerable spread between a heavy and a light connection in cumulative backlog messages
$\widehat{r}_u^{\text{ack}}$	Conservative client-observed ACK or message-processing rate for client u
τ_G	Acceptable additional drain-time skew used to derive a latency-driven backlog spread
ρ_N	Fraction of Θ_N used to set the default subscription-count imbalance tolerance
ρ_G	Fraction of $\beta\Theta_N$ used to set the default backlog-load imbalance tolerance
$V_u(t)$	Weighted local connection-imbalance score for client u
η_N, η_G	Nonnegative weights for subscription-count imbalance and backlog-load imbalance, with $\eta_N + \eta_G = 1$
γ_N, γ_G	Thresholds triggering subscription-count and backlog-based connection rebalancing

Table S5. Summary of Notations: Handoff and Experimental Endpoints

Symbol	Description
$\mathcal{K}_{\text{move}}, s_b, s_f$	Selected subscription set, barrier timestamp after source quiescence, and source telemetry-frame timestamp
$I_{c_h,k}(s_f)$, $\text{state}_{c_h,k}(s_f)$	Source-reported in-flight count and delivery state for subscription k at frame time s_f
c_p, i_h, i_l	Probe socket and Pod indices reached by c_h and c_l in the architecture figure
$t_k^{\text{act}}, t_k^{\text{send}}, t_k^{\text{ready}}$	Activation latency and submission/receipt timestamps in seconds; $t_k^{\text{act}} = t_k^{\text{ready}} - t_k^{\text{send}}$
$A^{\text{act}}(s)$	Number of receipt-confirmed activations by elapsed time s in seconds
$t_c^{\text{open}}, t_c^{\text{exp}}, T_{\text{TTL}}, J_c$	Connection opening and nominal expiry timestamps, TTL in seconds, and integer jitter in minutes
d_c, h_c, r_c	Disconnect, replacement-CONNECTED, and full binding-restoration timestamps in seconds
$T_c^{\text{conn}}, T_c^{\text{restore}}$	Transport reconnect and full restoration durations, measured from d_c
$T_u^{\text{handoff}}, T_u^{\text{drain}}$	Measured total handoff and source-drain durations for client u in seconds
$C_0, I_c(s), F(s)$	Initial trial connection pool, unrestored-connection indicator, and affected fraction at elapsed time s

Continued on next page

Table S5. Summary of Notations: Handoff and Experimental Endpoints (continued)

Symbol	Description
D_{obs}, A_{conf}	Recorded delivery attempts and attempts with confirmed ACK receipts
$B(t)$, AUC_B	Total backlog in the evaluated population and its normalized trapezoidal area over T intervals
$Y_{\pi,j,v}$, d_v, $D_{j,v}$	Episode mean for policy π, episode j, metric v; benefit direction; and signed paired difference
RB_v, p_H, d_z	Relative benefit in percent, Holm-adjusted p-value, and paired standardized mean difference
J, R_ℓ^{train}	Number of logged training transitions and the reward recorded for transition ℓ
M^{ep}, n_{ep}	Migration count in one episode and number of sampled episodes, distinct from client count m and Pod count n
$W_1, b_1, W_2, b_2, h(o)$	Weights and biases of the 21–32–6 Q-network, and its single ReLU hidden representation
$\mathbf{q}_\theta(o)$	Vector of online Q-values, one per action in $\mathcal{A}$, with $o = o_u(t)$

C Detailed Runtime Algorithms and Checkpoint Persistence

The main paper gives the shared control-loop summary. The following material retains the action-contract details, checkpoint behavior, sizing rules, executors, and full DQL algorithm.

C.1 Action Contract

The scale-up action is one logical option even though its guarded executor can contain several reassociations. After the new socket is ready, the executor repeatedly applies the same receipt-gated a^{rebN} primitive while $\max_c N_{u,c}(t) > \Theta_N$ and an eligible move exists. It records the opened session, completed move count, final maximum count, elapsed time, and whether the target was restored. If opening fails, the unused socket is removed. If a later reassociation fails, the completed receipt-gated moves remain valid and the option reports failure rather than claiming an atomic rollback of the whole sequence. New checkpoints carry an action-contract version. Full-action execution rejects the older open-only meaning of $a^{\text{scale+}}$. An explicit rebalancing-only allowlist may load a legacy checkpoint because it disables both scaling actions. This is the profile used in the October loaded cluster trial. Table S13 separates the evaluated contracts; the legacy checkpoint is not evidence for learned cap-restoring scale-up.

C.1.1 Persistent Learning and Frozen Evaluation. Learning is retained between runs through two checkpoint types. A research checkpoint stores both networks' weights and biases, replay transitions and their next-state action masks, the random-generator state, the exploration rate, the training-update count, and the learner configuration. Restoring this checkpoint continues the learner from its saved state rather than initializing new weights. Simulator continuation starts fresh workload episodes; the checkpoint does not restore an interrupted Kubernetes deployment or simulator trajectory. Checkpoints also retain the action-contract version so that replay from incompatible action meanings is not silently combined.

Each eligible transition is recorded before its model update, using the action actually applied and the subsequent observed reward. A masked decision or shadow recommendation is not evidence that a migration occurred; shadow execution contributes only its applied stay action. Stale or incomplete telemetry interrupts the learning trajectory

instead of joining observations across the missing interval. An append-only transition record and checkpoint hashes preserve a traceable history of later training.

A deployment checkpoint is a separate frozen copy: exploration is disabled, network updates are disabled, and replay is not modified during evaluation. The evaluation records observations and outcomes without turning them into training data for that comparison. Further development uses separately designated training data and produces a new candidate, which requires its own validation before deployment. Thus, retained learning avoids restarting from zero without making a fixed-policy experiment adapt to its test outcomes.

The layer is designed to:

- Dynamically scale the number of parallel persistent transport connections
- Improve parallel processing for high-backlog subscriptions
- Maintain balanced subscription distribution across connections
- Prevent overload of individual connections

C.2 Adaptive Parallel Connection Scaling - Algorithm

C.2.1 Mathematical Model. Unlike server-level placement, which changes the pod reached by a connection, connection scaling changes the amount of parallelism available to a client. In subscription-driven messaging systems, too few persistent connections can force many high-backlog subscriptions through the same processing path, increasing latency and delaying acknowledgements.

The algorithms use the same backlog $b_{u,k}(t)$ and connection subscription count $N_{u,c}(t)$ as the formal model. For compactness, the single-client scaling counts below suppress the fixed client index u.

Let the client-side subscription population be

$$N_{\text{sub}}(t) = |\mathcal{K}_u(t)|, \tag{S5}$$

where $\mathcal{K}_u(t)$ is the set of active subscriptions maintained by client u at time t. Thus $N_{\text{sub}}(t)$ counts subscriptions, not messages.

Let the current connection count be

$$C(t) = |\mathcal{C}_u(t)|, \tag{S6}$$

where $\mathcal{C}_u(t)$ is the set of active persistent transport connections used by client u for message delivery at time t. Hence $C(t)$ is the current number of usable STOMP/TCP/WebSocket-style delivery connections before the scaling decision is applied. The retained statistics-only probe is not counted in $C(t)$ unless it is promoted into a persistent delivery connection.

The desired count $C^{\star}(t)$ can differ from the amount executed in one DQL decision. Algorithm S1 computes the desired parallelism, whereas one $a^{\text{scale+}}$ option adds at most one connection. It then moves whole subscriptions until no active connection exceeds Θ_N, or until no eligible move remains. A subscription has exactly one active delivery binding outside the receipt-gated handoff interval; this mechanism does not split one subscription queue across several connections. Using the requirements defined below, the workflow abbreviates $C_{\text{back}}(t) = \max_{k \in \mathcal{K}_u(t)} C_k(t)$ and $C^{\star}(t) = \max\{C_{\text{sub}}(t), C_{\text{back}}(t)\}$ for a nonempty subscription set.

We define:

- Θ_N : maximum desired subscription count per connection
- β : backlog threshold

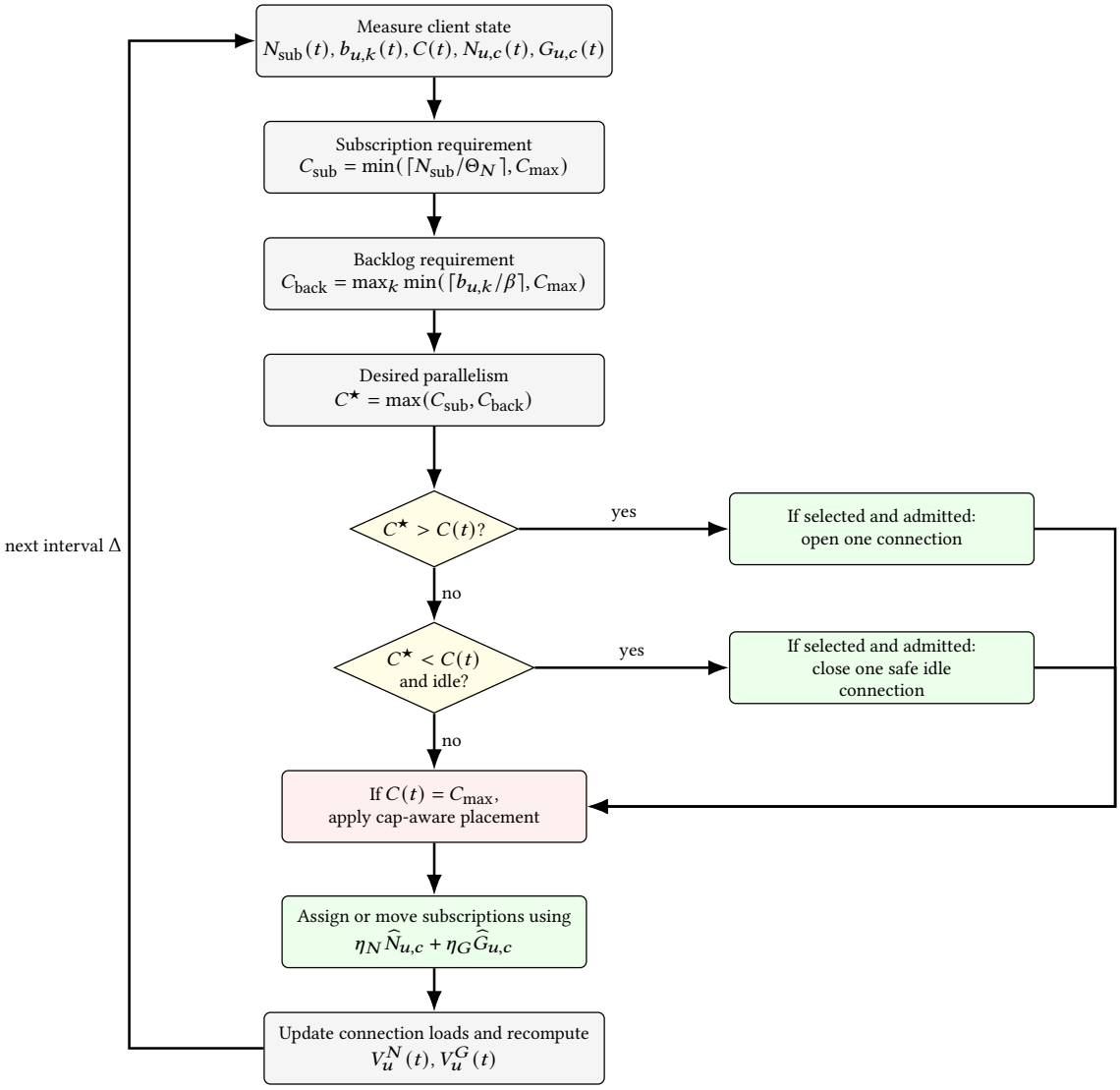


Fig. S1. Adaptive connection-scaling workflow. The client first computes the connection requirement induced by subscription density and the requirement induced by per-subscription backlog. The larger requirement determines the desired number of persistent connections, bounded by $C_{\max}$. If the cap is reached, the client assigns or moves subscriptions using a two-dimensional score that combines subscription-count pressure and backlog pressure. One DQL $a^{\text{scale+}}$ decision creates at most one connection and then restores the subscription target through bounded receipt-gated reassociation.

- $C_{\max}$: maximum allowable connections

The complete notation reference is collected in Appendix B.

C.2.2 Subscription-Based Scaling. As the number of subscriptions increases, each connection must handle more message streams. To prevent overload, the desired number of connections is defined as:

$$C_{\text{sub}}(t) = \min\left(\left\lceil \frac{N_{\text{sub}}(t)}{\Theta_N} \right\rceil, C_{\max}\right) \tag{S7}$$

Here Θ_N is a configured subscription-density threshold: it is the maximum desired number of subscriptions assigned to one persistent connection under normal operating conditions. It is not learned by DQL and is not the DQL parameter vector θ used in $Q_\theta(o, a)$.

If:

$$C_{\text{sub}}(t) > C(t)$$

the desired count exceeds the current count. The policy may request one admissible scale-up action; the sizing calculation itself opens no sockets.

The count rule bounds mean subscription density when sufficient capacity is available. A per-connection bound additionally requires the reassociations to complete; the executor records whether that target was restored.

The sizing rule and the reassociation executor therefore have separate roles: one requests enough sockets, and the other redistributes eligible subscriptions.

C.2.3 Max-Cap Distribution Strategy. When:

$$C(t) = C_{\max}$$

further scaling is restricted. In this scenario, newly arriving subscriptions are assigned using load-aware distribution.

Let:

$$N_{u,c_j}(t) \tag{S8}$$

denote the number of subscriptions assigned to connection c_j.

New subscriptions are assigned to the connection with the lowest combined subscription-count and backlog pressure:

$$c^{\text{place}} = \arg\min_{c_j \in C_u(t)} \left(\eta_N \widehat{N}_{u,c_j}(t) + \eta_G \widehat{G}_{u,c_j}(t)\right) \tag{S9}$$

This greedy balancing strategy avoids assigning new subscriptions to a connection that is already heavy either in subscription count or cumulative subscription backlog. Here c^{place} is the selected placement connection, while $c^\star$ is the connection evaluated for migration. The hats on N_{u,c_j} and G_{u,c_j} denote normalized loads, not the normalized variances $\widehat{V}_u^N$ and $\widehat{V}_u^G$.

C.2.4 Backlog-Based Scaling. Subscription backlog signals queued work, but its cause may lie in Pod service, client acknowledgement capacity, or shared queue bandwidth. Scaling is useful when additional connections can reach otherwise unused service capacity.

For subscription k, let backlog be:

$$b_{u,k}(t) \tag{S10}$$

If:

$$b_{u,k}(t) > \beta$$

the configured backlog-pressure threshold has been crossed.

The required number of connections is:

$$C_k(t) = \min\left(\left\lceil \frac{b_{u,k}(t)}{\beta} \right\rceil, C_{\max}\right) \tag{S11}$$

If:

$$C_k(t) > C(t)$$

backlog suggests additional parallelism. The selector still decides whether to execute a bounded scaling action under the current guards.

The additional capacity can then receive eligible whole subscriptions from loaded connections. Subscription k itself remains active on exactly one connection; its queue is not partitioned by this mechanism. Thus backlog-based scaling can improve parallelism across a set of subscriptions, but it cannot parallelize the delivery of one indivisible subscription queue by itself.

C.2.5 Scaling Objective. The objective of adaptive parallel connection scaling is to minimize backlog while respecting connection constraints:

$$\min \sum_{k \in \mathcal{K}_u(t)} b_{u,k}(t) \tag{S12}$$

subject to:

$$C(t) \le C_{\max} \tag{S13}$$

This represents a constrained parallel resource allocation problem.

Algorithm S1: Desired Connection Count (Sizing Only)

Input : Subscription set $\mathcal{K}_u(t)$, backlogs $b_{u,k}(t)$; thresholds Θ_N, β; hard cap $C_{\max}$
Output : Desired count $C^\star(t)$; no connection mutation

$N_{\text{sub}}(t) \leftarrow |\mathcal{K}_u(t)|$;
$C_{\text{sub}}(t) \leftarrow \min\{\lceil N_{\text{sub}}(t)/\Theta_N \rceil, C_{\max}\}$;
$C_{\text{back}}(t) \leftarrow 0$;
foreach $k \in \mathcal{K}_u(t)$ **do**
  $C_k(t) \leftarrow \min\{\lceil b_{u,k}(t)/\beta \rceil, C_{\max}\}$;
  $C_{\text{back}}(t) \leftarrow \max\{C_{\text{back}}(t), C_k(t)\}$;
$C^\star(t) \leftarrow \max\{C_{\text{sub}}(t), C_{\text{back}}(t)\}$;
return $C^\star(t)$ as decision context for the selector;

C.3 Connection-Level Load Rebalancing - Algorithm

C.3.1 Motivation. Adaptive connection scaling increases processing capacity as subscription count or backlog grows. Scaling alone, however, does not guarantee an even workload. New subscriptions, changing backlogs, and scaling decisions made at different times can leave some connections with far more subscriptions than others.

Such imbalance leads to:

- Uneven CPU utilization
- Increased processing latency on overloaded connections
- Underutilized parallel capacity on lightly loaded connections

Therefore, a dedicated rebalancing mechanism is required to maintain fairness and stability across scaled connections.

C.3.2 Load Distribution Model. Connection load is represented by two variables. For connection c, $N_{u,c}(t)$ is the number of subscriptions assigned to the connection, and $G_{u,c}(t)$ is the cumulative backlog of those subscriptions. The rebalancing controller therefore monitors both $V_u^N(t)$ and $V_u^G(t)$ as defined in the problem formulation.

High $V_u^N(t)$ means that subscription-management work is skewed across the client's persistent connections. High $V_u^G(t)$ means that queue pressure is skewed even if subscription counts look balanced. Either condition can degrade delivery stability.

C.3.3 Rebalancing Objective. The local objective of connection-level rebalancing is:

$$\min \left(\eta_N \widehat{V}_u^N(t) + \eta_G \widehat{V}_u^G(t) \right) \tag{S14}$$

subject to:

$$1 \leq |C_u(t)| \leq C_{\max}. \tag{S15}$$

This objective keeps both subscription-count skew and backlog skew under control.

C.3.4 Guarded Connection-Level Rebalancing Strategy. The two rebalancing templates have separate admissibility conditions:

$$\widehat{V}_u^N(t) > \gamma_N \quad \text{or} \quad \widehat{V}_u^G(t) > \gamma_G. \tag{S16}$$

The DQL policy, rather than Algorithm S2, decides whether the current observation calls for $a^{\text{reb}N}$, $a^{\text{reb}G}$, or another action. If it selects $a^{\text{reb}N}$, the executor moves one eligible subscription from the connection with largest $N_{u,c}(t)$ to the connection with smallest $N_{u,c}(t)$. If it selects $a^{\text{reb}G}$, the executor chooses the subscription whose reassociation from a high-backlog connection to a low-backlog connection produces the largest predicted decrease in $V_u^G(t)$. The corresponding threshold is then applied as a deterministic deployment guard. At most one bounded reassociation is executed in a control interval. This limits repeated back-and-forth moves and links the next reward to a clear action.

Algorithm S2 carries out the selected action; it is not a second policy. In particular, it never compares $\widehat{V}_u^N(t)$ with $\widehat{V}_u^G(t)$ to decide which objective will dominate. These normalized quantities can have different observed distributions even though both are bounded. The selected DQL template is the stored action, while the reward and next observation reflect the executor outcome, including a guarded no-op. The learned value therefore accounts for whether an action can be executed, without introducing a second rebalancing policy.

C.4 DQL-Based Rebalancing and Placement Algorithm

This section presents the client-driven DQL controller under the observability constraints described above. Each client keeps a statistics cache with two sources. Active connections report their current pod's tuple every $\Delta = 10$ seconds. An authenticated, statistics-only probe provides the other source. The load balancer assigns its endpoint when the socket is established. The client retains that socket and candidate-pod identity until promotion, expiry, or recycling while

Algorithm S2: Guarded Executor for DQL-Selected Connection-Level Rebalancing

Input : Selected template $a_u(t) \in \{a^{\mathrm{reb}N}, a^{\mathrm{reb}G}\}$; connections $C_u(t)$; subscription counts $N_{u,c}$; connection backlogs $G_{u,c}$; thresholds γ_N, γ_G

Output : Updated assignment, or the unchanged assignment when the selected template is inadmissible

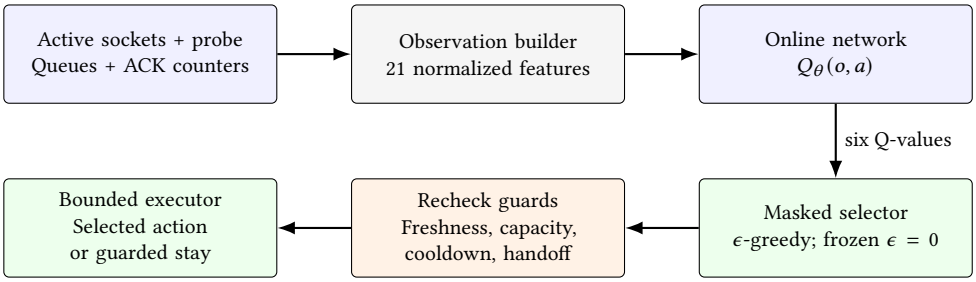
Compute $\widehat{V}_u^N(t)$ and $\widehat{V}_u^G(t)$;
if $a_u(t) = a^{\mathrm{reb}N}$ **then**
  if $\widehat{V}_u^N(t) \le \gamma_N$ **then**
    return unchanged assignment ; // Guarded no-op
  end if
  $c_h \leftarrow \arg\max_{c \in C_u(t)} N_{u,c}(t)$;
  $c_l \leftarrow \arg\min_{c \in C_u(t)} N_{u,c}(t)$;
  Select an eligible movable subscription k from c_h using a stable tie-break;
else
  if $\widehat{V}_u^G(t) \le \gamma_G$ **then**
    return unchanged assignment ; // Guarded no-op
  end if
  $c_h \leftarrow \arg\max_{c \in C_u(t)} G_{u,c}(t)$;
  $c_l \leftarrow \arg\min_{c \in C_u(t)} G_{u,c}(t)$;
  Select eligible k on c_h maximizing the predicted decrease in $V_u^G(t)$;
end if
if $c_h = c_l$ *or no eligible* k *exists* **then**
  return unchanged assignment ; // Guarded no-op
end if
Reassociate k from c_h to c_l using the guarded executor;
Update $N_{u,c_h}, N_{u,c_l}, G_{u,c_h}, G_{u,c_l}$;
return updated assignment;

periodically obtaining the tuple in (6 of the main paper). The controller then uses the observation vector in (23 of the main paper) to select one of the action templates in (24 of the main paper).

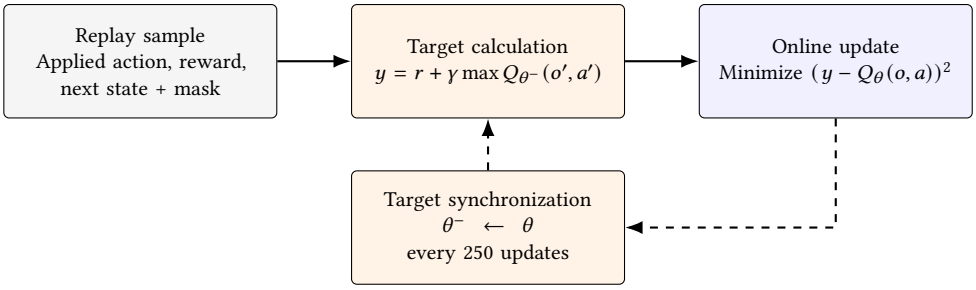
In Figure S3, o abbreviates $o_u(t)$ and $\theta = (W_1, b_1, W_2, b_2)$. There are 902 trainable scalar parameters in each network. The output order is $(a^{\mathrm{stay}}, a^{\mathrm{reb}N}, a^{\mathrm{reb}G}, a^{\mathrm{scale+}}, a^{\mathrm{scale-}}, a^{\mathrm{mig}})$. Grouping features by meaning in (23 of the main paper) does not introduce additional neural layers. Table S6 gives the evaluated v3 configuration; Appendix D gives the exact input order and normalization.

Table S6. Configuration of the v3 DQL policy used in the reported holdout, scale-out simulator, and October live trial. Later learner variants are reported separately in Appendix L.

Setting	Value
Network; optimizer	Dense 21–32–6; ReLU hidden, linear output; mini-batch gradient descent
Discount; learning rate	$\gamma = 0.99$; $\alpha = 0.001$
Replay; batch; target copy	Uniform replay, capacity 5,000; batch 16; copy every 250 updates
Bootstrap; loss	Target-network maximum over admitted actions; squared TD error
Training budget	10,000 transitions per candidate; 9,985 updates after batch warm-up
Training exploration	$\epsilon_0 = 1$, multiplier $\exp(\log(0.05)/10{,}000)$; floor 0.05
Reward weights $(\lambda_1, \ldots, \lambda_7)$	$(0.20, 0.15, 0.20, 0.15, 0.10, 0.05, 0.15)$
Pressure weights $(\omega_X, \omega_B, \omega_H, \omega_R)$	$(0.30, 0.25, 0.20, 0.25)$
Imbalance weights	$\eta_N = \eta_G = 0.5$
Frozen evaluation	$\epsilon = 0$; no model or replay updates; checkpoint hash retained

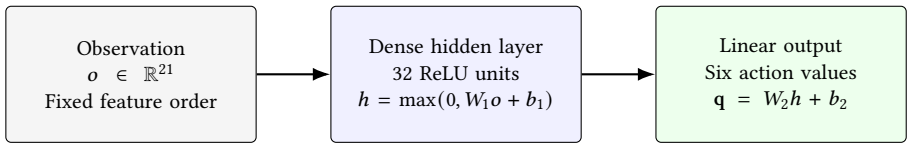


(a) Runtime decision path. The target network is not on this path.



(b) Training path for the evaluated v3 learner. Frozen evaluation disables updates.

Fig. S2. DQL selection and learning are separate from the shared guarded executor. Solid arrows show runtime or training data flow; dashed arrows copy parameters to the target network. Only the online network ranks live actions. The target network stabilizes the next-state value used during training.



$$W_1 : 32 \times 21, \quad b_1 : 32, \quad W_2 : 6 \times 32, \quad b_2 : 6$$

Fig. S3. The evaluated Q-network: 21 input features, one fully connected 32-unit ReLU layer, and six linear outputs. Each arrow denotes a dense matrix operation, not a sparse connection between individual neurons. There are no separate learned feature encoders. The online and target networks have this same architecture and different parameter-update roles.

C.4.1 Reward-Based Learning. For every observed pod $i \in O_u(t)$, the client computes pod pressure $P_i(t)$ from (18 of the main paper). This pressure explicitly penalizes both pod subscription count and pod cumulative backlog. The subscription-count term captures the observed reliability degradation of pods with excessive subscription-management load, while the backlog term captures queue accumulation. The threshold H_i is used only as a capacity-normalized pressure term; it is not interpreted as the actual client-side drain rate.

After action execution, the client computes reward $R_u(t)$ using (22 of the main paper). This reward includes local backlog, backlog growth, local connection imbalance, migration cost, connection cost, pod pressure, and effective client-side throughput. Hence the learned policy is trained using information that the client can actually observe at runtime.

C.4.2 Exploration, Hysteresis, and Migration. Let $\mathcal{A}_u^{\mathrm{adm}}(t) \subseteq \mathcal{A}$ contain only the templates that pass the current freshness, capacity, cooldown, and handoff guards. The DQL policy uses epsilon-greedy exploration over this masked set:

$$a_u(t) = \begin{cases} \mathrm{Uniform}(\mathcal{A}_u^{\mathrm{adm}}(t)), & \text{w.p. } \epsilon, \\ \arg\max_{a \in \mathcal{A}_u^{\mathrm{adm}}(t)} Q_\theta(o_u(t), a), & \text{w.p. } 1-\epsilon. \end{cases} \tag{S17}$$

The exploration probability decreases during training. Frozen validation and deployment policies use $\epsilon = 0$. This prevents unreviewed exploratory changes during evaluation. Online exploration would require a separate safety study. Migration is executed only when the selected action is a^{mig} and the retained probe candidate improves the estimated value by a hysteresis threshold:

$$Q_\theta(o_u(t), a^{\mathrm{mig}}) > Q_\theta(o_u(t), a^{\mathrm{stay}}) + \delta. \tag{S18}$$

This threshold prevents oscillatory migrations caused by small or stale statistical differences. It does not choose the probe endpoint: the Kubernetes load balancer already fixed that endpoint when the probe socket was opened. Passing this value test is necessary but not sufficient. The executor also checks telemetry freshness, target capacity, source–target separation, cooldown, and the absence of a conflicting handoff. As a result, most probes can remain telemetry-only and may expire or be recycled without carrying a subscription.

$$\begin{aligned} \mathrm{computeReward}(i, u, t) = R_u(t) = &- \lambda_1 \widehat{B}_u - \lambda_2 \widehat{\mathrm{Grow}}_u \\ &- \lambda_3 P_i - \lambda_4 V_u \\ &- \lambda_5 M_u - \lambda_6 \widehat{C}_u + \lambda_7 \widehat{T}_u \end{aligned} \tag{S19}$$

The epsilon-greedy rule is therefore used as the exploration mechanism inside DQL, not as the main learning model. Unlike a stateless bandit controller, DQL can learn delayed effects. Scale-up with subscription restoration has an immediate connection and reassociation cost but may later reduce backlog growth. Migration may have an immediate penalty but later avoid an unreliable pod with too many subscriptions. A rebalancing move may help only after queues have had time to drain.

C.5 Hierarchical Optimization Structure

The resulting control structure has two complementary levels:

(1) **Server-Level Optimization:** The DQL controller selects migration only when the observed pod statistics and local client state suggest that moving to a candidate pod is likely to improve long-term reward after migration cost.

(2) **Connection-Level Optimization:** The same policy decides when to rebalance subscriptions, increase or decrease connection parallelism, or leave the current assignment unchanged.

This hierarchical design enables simultaneous optimization of:

- Inter-server load distribution
- Intra-client parallelism
- Subscription fairness
- Backlog reduction

Algorithm S3: DQL-Based Client-Side Persistent Transport Rebalancing

Input : Replay buffer $\mathcal{D}$; online network Q_θ; target network Q_{θ^-}
Observed pod set $\mathcal{O}_u(t)$ and its cached tuples $y_i(t)$; retained authenticated probe and endpoint $i_u^{\text{probe}}(t)$
Exploration probability ϵ; discount factor γ; migration threshold δ; target Θ_N; hard cap $C_{\max}$
Output : Adaptive rebalancing, scaling, and migration decisions

$t \leftarrow 0$; use the active delivery pool and one authenticated probe;
Load the declared checkpoint; initialize networks/replay only for new training;
while *client is active* **do**
  Update the pod cache from active and retained-probe telemetry;
  Measure per-subscription backlogs, connection loads, throughput, and acknowledgements;
  Construct observation $o_u(t)$ using (23 of the main paper);
  Construct the masked admissible set $\mathcal{A}_u^{\text{adm}}(t)$ from freshness, capacity, cooldown, and handoff guards;
  $a_u(t) \leftarrow \epsilon$-greedy$(Q_\theta, o_u(t), \mathcal{A}_u^{\text{adm}}(t))$;
  if $a_u(t) \in \{a^{\text{reb}N}, a^{\text{reb}G}\}$ **then**
    Instantiate the selected template using Algorithm S2;
  else
    if $a_u(t) = a^{\text{mig}}$ **then**
      if *probe and handoff guards pass, including the hysteresis test* **then**
        Promote the same retained probe and execute the receipt-gated handoff in Section 3.1 of the main paper;
        Open a new statistics-only probe through the load balancer after handoff;
      else
        Record a guarded no-op; preserve source ownership and probe role;
      end if
    else
      if $a_u(t) = a^{\text{scale+}}$ **then**
        if *capacity and subscription-demand guards still pass* **then**
          Open one connection; apply receipt-gated $a^{\text{reb}N}$ moves until $\max_{c \in C_u(t)} N_{u,c}(t) \le \Theta_N$ or no move exists;
          Record move count and target-restored status;
        else
          Record a guarded no-op;
        end if
      else
        if $a_u(t) = a^{\text{scale}-}$ **then**
          Close one safe idle connection after rechecking its guard;
        else
          Preserve the current topology for a^{stay};
        end if
      end if
    end if
  end if
  Observe reward $R_u(t)$ from (22 of the main paper) and next observation $o_u(t+1)$;
  Log the applied action, reward, next observation, and guard outcome;
  if *training is enabled and the transition is valid* **then**
    Store the applied-action transition and next-state mask in $\mathcal{D}$;
    Update from a mini-batch using (26 of the main paper); synchronize the target when due;
  end if
  Wait Δ; $t \leftarrow t + 1$;
end while
return policy and recorded action outcomes;

The learned component does not create arbitrary subscription assignments. It chooses bounded action templates that the client can execute and check against deployment guards.

D Evaluated Features and Policy Settings

The frozen network consumes the ordered 21-element vector in Table S7. For nonnegative x and positive reference r, the implementation uses $\mathrm{clip}(x/r) = \min(1, x/r)$ and $\mathrm{lognorm}(x; r) = \min(1, \log(1+x)/\log(1+r))$. These are numerical scaling rules, not drain-time estimates. A pressure feature can exceed one through the quadratic reliability term even though its count, backlog, and capacity components are clipped.

Table S7. Exact input order of the evaluated 21–32–6 network. Indices are one-based; "mean" and "maximum" refer to active client connections.

Indices	Features, in order
1–2	Log-normalized client backlog; log-normalized positive backlog growth
3–5	Mean and maximum of $\mathrm{clip}(N_{u,c}/\Theta_N)$; $\widehat{V}_u^N$
6–9	Mean and maximum of $\mathrm{lognorm}(G_{u,c}; B_{\mathrm{ref}})$; $\widehat{V}_u^G$; combined imbalance V_u
10–11	$\lvert C_u \rvert / C_{\max}$ clipped at one; client throughput divided by the declared throughput reference and clipped at one
12	Active-Pod pressure, weighted by this client's subscription counts on each observed Pod; unweighted mean if the count is zero
13–15	Probe pressure; clipped probe subscription count relative to X_{safe}; clipped probe backlog relative to B_{ref}
16–18	Probe-present indicator; probe endpoint absent from active endpoints; cooldown-ready indicator
19–21	Maximum active telemetry/backlog age ratio; probe age ratio; previous-action-was-migration indicator

Pod-wide values are deduplicated when several sockets reach one Pod. Client backlog sums each known subscription once. When per-subscription values are absent, the builder uses the active-connection backlog aggregates. Age ratios are clipped at one. Missing probe data sets its pressure and age features to one and its present indicator to zero; the independent validity checks prohibit migration. Coverage and freshness checks are execution gates, not additional neural inputs.

For the simulator adapter, B_{ref} is the declared initial tiered backlog when it is positive, otherwise $\max(1, \beta|\mathcal{K}_u|)$. The growth reference equals B_{ref}; $V_{\mathrm{ref}}^N = (0.2\Theta_N)^2$ and $V_{\mathrm{ref}}^G = (0.2B_{\mathrm{ref}})^2$. The safe and hard subscription thresholds are $2\Theta_N$ and $\max(2\Theta_N + 1, \mathrm{round}(2.5\Theta_N))$. The throughput reference is three times the configured base client ACK rate. These constants are scenario inputs, not fitted from holdout outcomes. The service-model pressure used to generate simulator throughput is distinct from this observation-builder pressure, as described in Section 7.1 of the main paper.

The October loaded deployment uses $\Theta_N = 250$, $C_{\max} = 5$, $B_{\mathrm{ref}} = 100{,}000$, $V_{\mathrm{ref}}^N = 2{,}500$, $V_{\mathrm{ref}}^G = 400{,}000{,}000$, $X_{\mathrm{safe}} = 800$, and $X_{\mathrm{hard}} = 1{,}000$. Its throughput reference is 1,000 events/s, telemetry deadline 180 seconds, minimum backlog coverage 0.8, and decision interval 10 seconds. The four active connections fit within the configured cap. Both scaling actions are disabled by the deployment profile. Migration requires projected pressure improvement above 0.01 and Q-value margin above $\delta = 0.05$. These live normalization constants differ from the simulator's; the live trial evaluates execution, not a matched cross-domain reward gain.

The mixed-workload *Heuristic* checks admitted actions in this order: subscription variance above 0.45; backlog variance above 0.45; normalized backlog above 0.55 for scale-up; active-minus-probe pressure above 0.18 for migration; connection utilization below 0.25 for scale-down; otherwise stay. The scale-out *guarded-greedy* selector instead requests migration whenever its shared guard admits it. The *Scripted* timing controls use their declared random schedules. None receives the DQL network's action values.

E Extended Cloud and Edge Literature

E.1 Cloud, Microservice, and Load-Balancing Literature

Recent cloud resource-management studies emphasize that adaptive schedulers must operate under workload uncertainty, heterogeneous resources, and competing objectives such as latency, throughput, energy, and utilization [23]. Reviews of DRL-based cloud control identify the same practical issues that matter here: reward design, non-stationary demand, and operational deployment [35]. DRL has been used for algorithm selection in hierarchical cloud scheduling [36], feedback-control-based cloud resource allocation [8], multi-objective virtual-machine placement [5], and multidimensional allocation in distributed data centers [32]. These works show how learned control can account for delayed effects and competing objectives. However, their controlled objects are usually tasks, virtual machines, jobs, or data-center resources. Our controlled objects are lower in the messaging stack: subscriptions, persistent client connections, and observed subscriber pods.

Cloud scheduling offers several distinct approaches, not all of them learning-based. Belgacem et al. use a spacing multi-objective antlion algorithm to optimize makespan and virtual-machine cost [2]. Ding et al. instead use Q-learning for energy-efficient task scheduling [14], while Chen et al. use actor–critic DRL for resource allocation in cloud data centers [9]. Collaborative scheduling addresses heterogeneous workflows [6]; workload-time windows provide temporal context for cloud-service allocation [7]. Li et al. apply DRL to energy-aware Spark scheduling [24]. Sanjalawe et al. provide a comprehensive review of AI-driven cloud job scheduling [30].

The controlled workload also matters. Dong et al. use DQN for manufacturing task scheduling [15], and Chen et al. study attention-based scheduling across manufacturing cloud and edge resources [10]. Cheng et al. introduce preemption into cost-aware cloud scheduling [11]; Liu et al. consider inter-region bandwidth and traffic costs for data-intensive jobs [25]. These papers supply scheduling context. Our problem instead requires subscription reassociation while retaining queue identity and delivery ownership across persistent connections.

Edge research is relevant here in two narrower respects: decentralized observations and explicit tradeoffs among resource costs. Iftikhar et al. provide a taxonomy of AI-based fog and edge computing [20], while Hortelano et al. survey RL-based computation offloading [17]. A concrete decentralized example is Nieto et al.'s per-device actor–critic offloading policy, which trades battery-related QoE against task-latency requirements [27]. Hou et al. explicitly formulate distributed offloading with partial global information [18]. Partial observability is therefore already addressed in earlier work. It is not, by itself, a new contribution of our controller.

Other offloading papers inform the choice of cost terms, rather than the transport handoff mechanism. Xu et al. optimize cloud-edge resource rental and allocation costs [33]. Huang et al. combine computation offloading with resource allocation in wireless-powered edge networks [19]. Pan et al. model dependent tasks and train an offloading scheduler using a modified PPO method [28]. Jayanetti et al. consider energy and execution time for precedence-constrained tasks [22]. Mitsis et al. incorporate price and risk into offloading decisions [26], whereas Cui et al. optimize caching and request routing for latency-sensitive content distribution [13]. Our migration and connection costs likewise make operational tradeoffs explicit, but their definitions and the subscription-ownership contract are specific to persistent messaging.

Microservice autoscaling provides a closer infrastructure comparison. Abdel Khaleq and Ra use reinforcement learning to control horizontal scaling from resource demand [1]. HephaestusForge learns microservice placement across Kubernetes clusters with multiple deployment objectives [31]. These approaches change service capacity or placement.

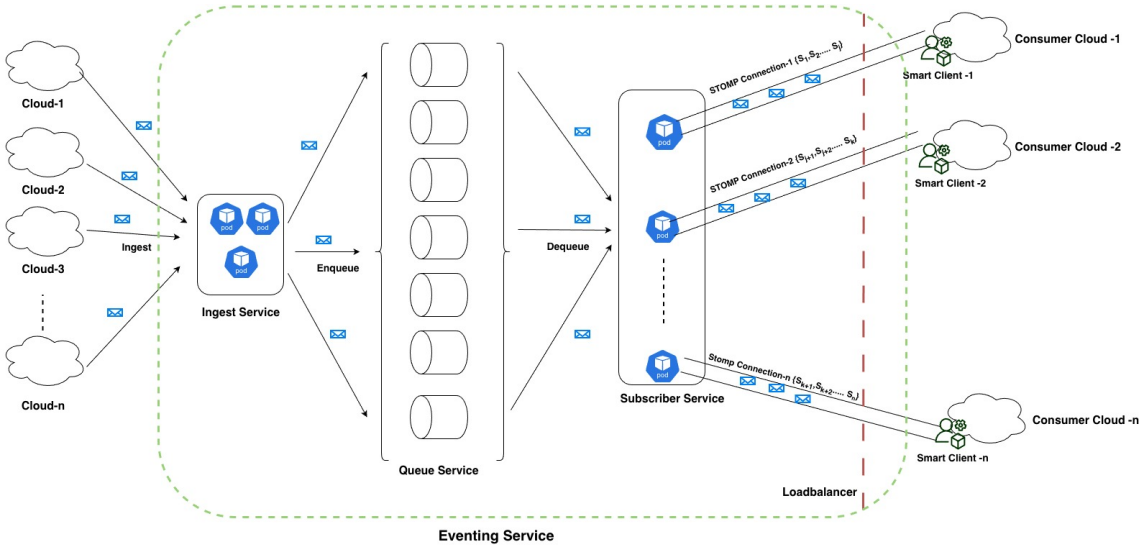


Fig. S4. System architecture with the DQL policy option shown. Producers send events through ingest services to durable subscription queues. Subscriber Pods deliver events over persistent connections. A client retains a load-balancer-assigned probe and promotes that same socket only after policy selection and execution checks. Greedy selection uses the same observation and actuation interfaces.

Our mechanism addresses the complementary client-side step: existing subscription-bearing sockets must reach the added capacity, and candidate promotion must account for both subscription and backlog pressure.

F System Model

Figure S4 illustrates the overall architecture of the event-driven messaging system considered in this work. The system consists of four major components: (1) event producers, (2) ingest and queue services, (3) subscriber services, and (4) distributed consumer clients connected via persistent subscription transport connections.

F.1 Event Producers and Ingest Service

Event producers run in independent cloud environments, shown as Cloud-1 through Cloud-n in Figure S4. They generate events asynchronously and send them to the ingest service, which is the system's entry point. The ingest service is implemented as a horizontally scalable set of service instances deployed as pods. Each ingest instance processes incoming events and enqueues them into the distributed queue service. The ingest service ensures reliable message intake and provides elasticity to handle fluctuating workloads.

F.2 Queue Service

The queue service acts as an intermediate buffering layer between event producers and subscribers. It provides durable storage and ensures reliable delivery of messages. Messages received from the ingest service are placed into queues, where they remain until consumed by subscriber services.The queue service supports parallel queues and can scale as event volume grows. Buffering lets producers and consumers work independently, without matching their processing rates.

F.3 Subscriber Service

The subscriber service is responsible for dequeuing messages from the queue service and delivering them to consumer clients. Similar to the ingest service, the subscriber service is deployed as multiple horizontally scalable instances. Each subscriber service instance maintains persistent transport connections with multiple consumer clients. These connections may use STOMP-over-TCP, raw TCP, WebSocket, or another long-lived subscription-bearing transport. They allow real-time event delivery and enable clients to subscribe to specific event streams. Subscriber service instances perform the following key functions:

- Dequeue messages from the queue service
- Maintain subscription state for connected clients
- Deliver messages to subscribed clients
- Track subscription load, backlog, and delivery rate

The subscriber service instances are deployed behind a load balancer, which distributes incoming client connection requests across available subscriber instances.

F.4 Load Balancer

The load balancer serves as the initial connection routing layer between consumer clients and subscriber service instances. It distributes client connections across available subscriber instances using traditional transport-level algorithms such as round-robin or least-connections.

However, the load balancer operates without visibility into application-level metrics such as subscription count, message backlog, or processing load. As a result, connection assignments may not reflect the true load distribution across subscriber instances. This limitation motivates the need for adaptive client-driven connection placement mechanisms.

F.5 Consumer Clients and Persistent Transport Connections

Consumer clients, referred to as Smart Clients in Figure S4, are deployed across multiple consumer cloud environments. These clients establish persistent subscription transport connections to subscriber service instances.

Each persistent transport connection can support multiple subscriptions. For client u, let $\mathcal{K}_u(t)$ be its subscription set and $C_u(t)$ its active delivery-connection set. The subscriptions assigned to connection $c \in C_u(t)$ are

$$\mathcal{K}_{u,c}(t) = \{k \in \mathcal{K}_u(t) : z_{u,k,c}(t) = 1\}. \tag{S20}$$

Here $z_{u,k,c}(t)$ is the binary assignment indicator defined in Section 4 of the main paper. Each subscription corresponds to a specific event stream or topic; k indexes a subscription, not a Pod.

The subscriber service maintains subscription state and delivers messages to clients based on these subscriptions. Because subscription counts and event rates may vary significantly across connections, the load imposed by each client connection is highly variable.

F.6 Connection Placement and Rebalancing Model

Let the set of subscriber service instances be denoted as:

$$\mathcal{S} = \{1, 2, \ldots, n\}. \tag{S21}$$

Each subscriber instance exposes the following dynamic statistics to active connections and probe connections:

$$y_i(t) = \big(p_i, X_i(t), B_i(t), H_i\big). \tag{S22}$$

where:

- p_i is the pod name
- $X_i(t)$ is the pod-level subscription count
- $B_i(t)$ is the cumulative backlog over all subscription queues served by the pod
- H_i is the configured maximum processing threshold, used only as a capacity-normalization signal

These statistics are exposed to clients through protocol-specific statistics frames, headers, or control messages. Actual drain behavior is measured from the client side using delivered, processed, and acknowledged message counts, because the effective drain rate also depends on client processing capacity.

Decentralized monitoring research similarly treats telemetry quality and monitoring overhead as coupled concerns. DEMon, for example, uses adaptive gossip-based monitoring in volatile edge environments [21]. Here the telemetry scope is deliberately smaller: active sockets and a retained probe provide sampled Pod observations, not a complete cluster view.

Clients maintain persistent connections and periodically evaluate alternative subscriber instances using a learning-based placement and rebalancing strategy. Based on observed load characteristics, clients may rebalance subscriptions across their own connections, scale connection parallelism, or migrate subscriptions to alternative subscriber instances.This adaptive mechanism enables the system to dynamically rebalance load across client connections and subscriber service instances while operating only on client-visible statistics.

F.7 System Characteristics

The system exhibits the following important properties:

- **Persistent connections.** Clients maintain long-lived STOMP-over-TCP, raw TCP, WebSocket, or equivalent subscription-bearing connections.
- **Subscription-Based Load:** Server load is determined primarily by subscription count and event rate, rather than connection count.
- **Dynamic Workload:** Event rates and subscription distributions change over time.
- **Distributed Architecture:** Subscriber services operate independently and scale horizontally.
- **Limited Load Balancer Visibility:** Load balancers lack application-level visibility into subscription load.

These characteristics make adaptive, client-driven connection placement particularly suitable for achieving optimal load balancing.

G Additional 20,000-Subscription Handoff Diagnostics

The deployment setup and aggregate results appear in the main paper. The following paired plot retains the within-run timing detail.

Figure S5 retains the pairing that a summary median cannot show. For client u, the interval between its two markers is $T_u^{\text{handoff}} - T_u^{\text{drain}}$, the measured non-drain remainder of the handoff; it is not an estimate of network latency alone. The box extends from the sample's first to third quartile, so the interquartile range describes the central half of the timings. All points, including the long-duration observations, remain visible. Longer handoffs coincide with longer

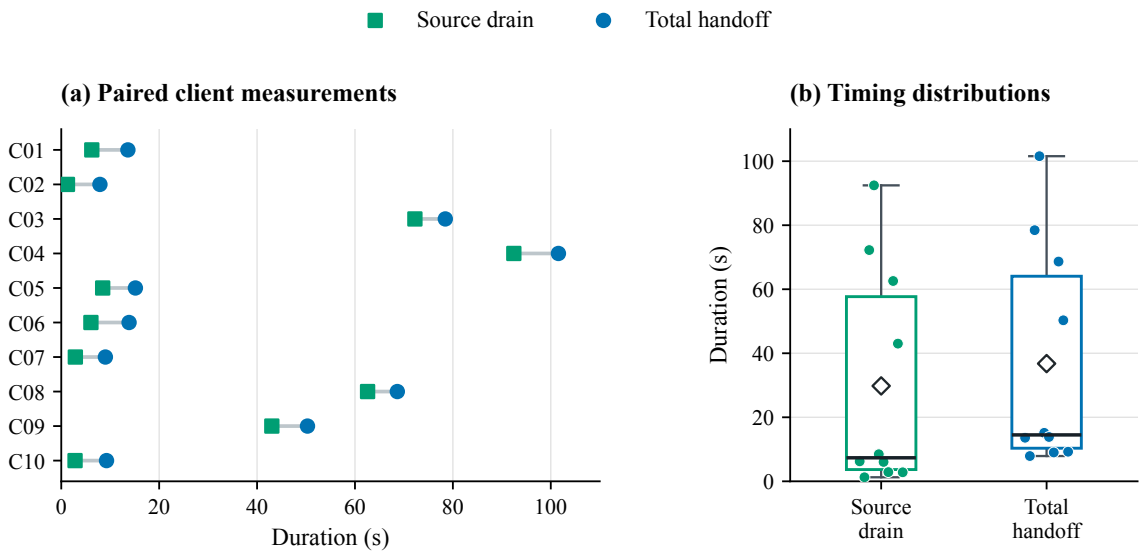


Fig. S5. Measured Oracle Cloud cluster migration timing. Left: paired source-drain and total-handoff durations for each of the ten client migrations. Right: quartile boxes with median lines, mean diamonds, and all ten observations; whiskers extend to observations within 1.5 interquartile ranges. These are ten migrations in one cluster run, not ten independent cluster trials.

source drains, consistent with the receipt-gated mechanism waiting for outstanding work. These are within-run timing distributions; their scope is the observed deployment, without a between-deployment confidence interval.

An independent collector retained 31 observations of each of the ten pods over approximately 2,422 seconds. All pods remained Ready, with unchanged pod, image, and process identities and no restart increase. These observations establish operational continuity over this observation period. Long-term availability is outside the measurement window.

H Concentrated Placement and Startup Under Load

We conducted a separate Oracle Cloud cluster experiment with an isolated subscriber Deployment and Service to examine what happens when persistent TCP connections remain on one Pod. In an initial 4,000-subscription static calibration, ten delivery connections and two probes terminated on one Pod. Adding three Ready Pods did not relocate these sockets: the source retained all twelve established connections, while each added Pod had none. The clients nevertheless received and acknowledged events while subscription activation was still progressing. Full activation took approximately 1,398.4 seconds; the completed run reconciled 137,008 observed deliveries with confirmed ACKs. This shows the difference between having server capacity and reaching it through existing persistent connections.

A later frozen-DQL test used 2,000 other subscriptions, divided equally among four event-type groups. One client attached five connections, each carrying 400 subscriptions, to the single source Pod. Three destination Pods became Ready before standby bindings were activated. The client used 32 processing workers without an artificial processing delay, ten message permits per subscription, and a window of 32 outstanding activation requests. Exact lifecycle records used batched SQLite transactions with FULL synchronization and a mandatory flush at measurement and closure boundaries. The earlier calibration used a different subscription population and recorder settings. The two timing records are therefore reported as separate experiments rather than a matched static-policy comparison.

Table S8. Measured isolated Oracle Cloud cluster connection-rebalancing qualification. These values describe one live run, distinct from the earlier 20,000-subscription deployment.

Measurement	Observed value
Available Pods / subscribed Pods before and after	4 / 1 and 3
Subscriptions / active connections / probe	2,000 / 5 / 1
Time to 50% / 90% / full activation (s)	238.066 / 388.139 / 426.305
Committed connection migrations / subscriptions moved	3 / 1,200
Source subscriptions before / after	2,000 / 800
Final subscribed-Pod populations	800, 800, 400
Handoff times, in execution order (s)	10.878, 10.992, 10.648
Source-drain waits, in execution order (s)	6.069, 6.174, 6.848
Delivered / processed / receipt-confirmed ACKs	73,639 / 73,639 / 73,639
Unresolved observed attempts at closure	0
Recorded ACK failures / duplicate candidates	0 / 0

The frozen controller committed three complete connection replacements, each moving 400 subscriptions through a retained probe. The source population fell from 2,000 to 800, a reduction of $(2{,}000 - 800)/2{,}000 = 60\%$. Two destination Pods acquired 400 and 800 subscriptions. The active-connection count remained five, and all 2,000 subscriptions were confirmed in the final topology after the two-minute hold. The migrations therefore changed which Pods served these subscriptions without increasing the client's active-connection count.

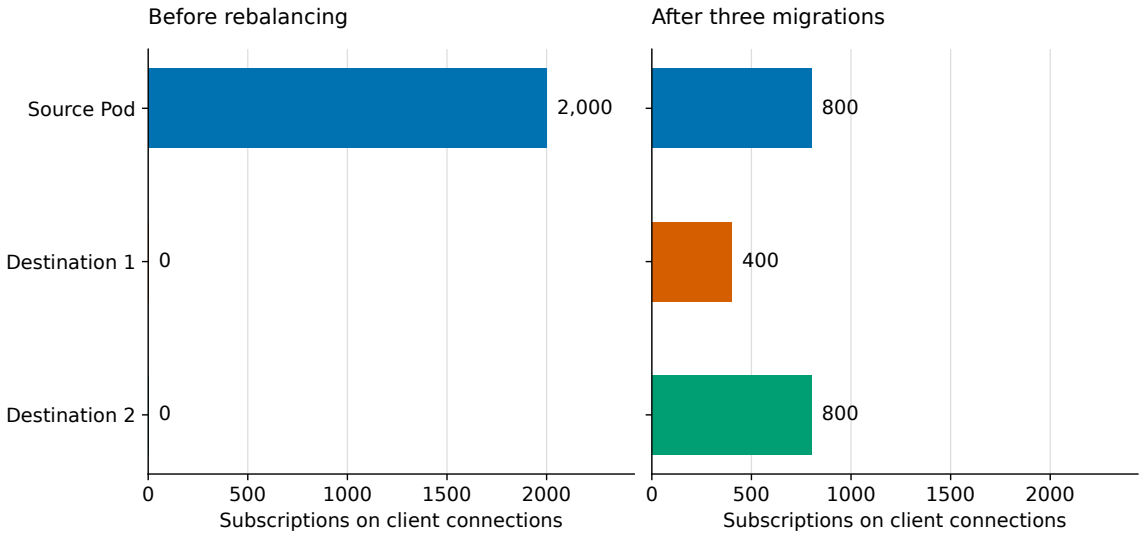


Fig. S6. Verified client subscription placement before and after three frozen-DQL connection migrations in the Oracle Cloud cluster. Bars aggregate the client's receipt-confirmed bindings by serving Pod. Initial zeros mean no bindings belonging to this client, not a measurement of every cluster subscription. Both panels account for the same 2,000 subscriptions and five connections.

Figure S6 shows the main operational effect: newly available capacity becomes reachable by moving subscription-bearing connections. It reports verified connection endpoints, rather than estimating changes between observations. A probe was first resampled because it did not offer an admissible improvement. Eleven later decision intervals selected

stay while backlog telemetry was incomplete; migration resumed when the observation checks passed. The online and target network parameters remained identical to the loaded checkpoint, while valid transitions were retained.

H.1 Activation and Delivery Coexistence

For subscription k, let t_k^{send} be activation submission time and t_k^{ready} its successful receipt time, relative to the start of activation. At elapsed time s in seconds, the cumulative completion count is $A^{\text{act}}(s) = \sum_{k=1}^{2000} \mathbf{1}\{t_k^{\text{ready}} \le s\}$, and $2000 - A^{\text{act}}(s)$ is the pending count. This separates attachment from confirmed activation and makes startup delay visible even while events flow. The 50th, 90th and 100th completion percentiles were 238.066, 388.139 and 426.305 seconds, respectively. No activation retry was needed in this run.

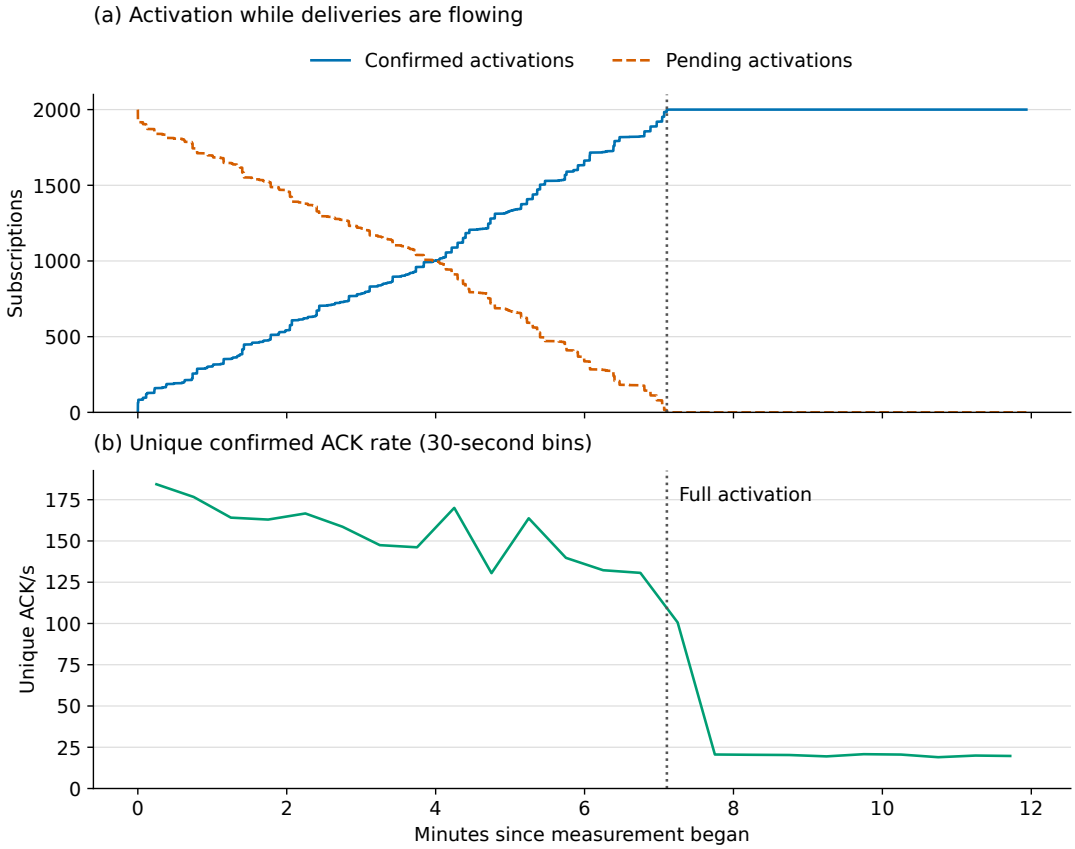


Fig. S7. Activation and acknowledged delivery in the isolated Oracle Cloud cluster run. Top: confirmed and pending activation counts. Bottom: distinct observed subscription–message pairs whose first successful ACK receipt occurs in each 30-second bin, divided by that bin's actual duration. The dotted line marks full activation. DQL decisions start after this line.

Figure S7 retains the startup interval rather than treating it as discarded warm-up. Of 73,587 distinct first ACKs inside the 716.017-second measurement window, 66,400 occurred before full activation. A further 52 attempts completed during shutdown, bringing final reconciled deliveries to 73,639. The rate curve describes consumption of existing queued work. It describes startup and workload drainage rather than constant-offered-load capacity. DQL begins after

full activation, so these measurements establish delivery during startup followed by redistribution. Activation time precedes the policy intervention.

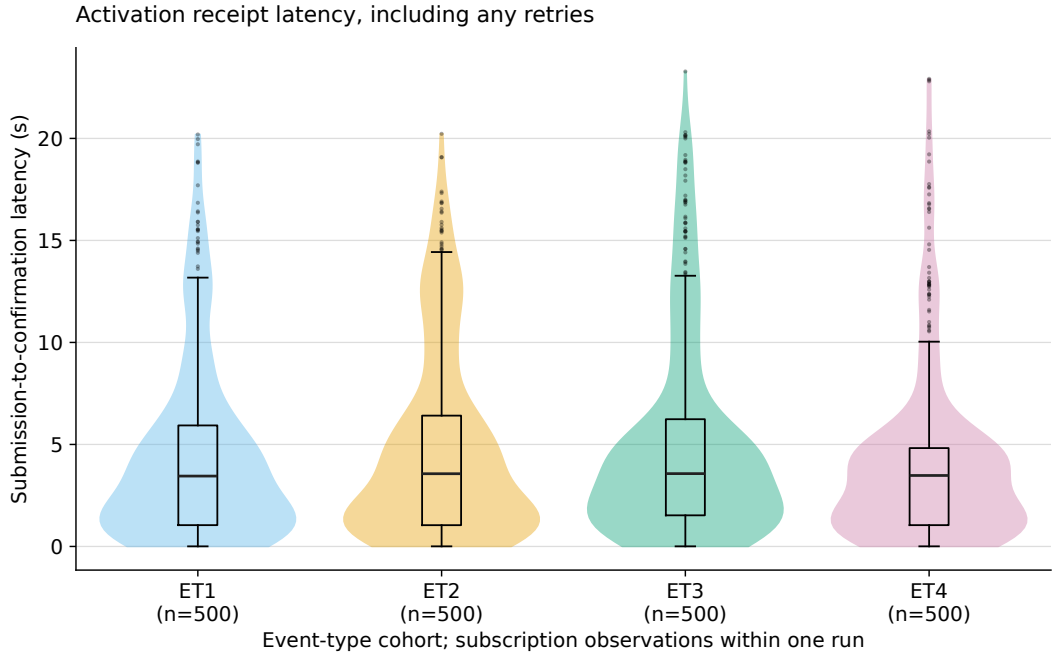


Fig. S8. Within-run activation latency $t_k^{\mathrm{act}} = t_k^{\mathrm{ready}} - t_k^{\mathrm{send}}$ for 500 subscriptions in each event-type cohort. Violin widths show smoothed empirical density; boxes show the interquartile range and median. Whiskers use 1.5 interquartile ranges, with observations beyond them retained as points. These are subscription observations in one run, not independent cluster trials.

Figure S8 distinguishes an individual request's latency from the time to activate the complete population. Cohort medians were 3.443, 3.560, 3.567 and 3.476 seconds; nearest-rank 99th percentiles were 18.805, 16.907, 20.072 and 20.038 seconds. The subscriptions share upstream services and scheduling conditions, so their observations are dependent. The distributions describe within-run variation, without estimating a between-policy confidence interval. A concurrent thread snapshot found fourteen subscription handlers waiting for upstream transition confirmation. This locates the observed startup bottleneck in upstream confirmation waits; the snapshot does not establish CPU saturation.

The complete ledger covers observed delivery attempts, with publisher-to-Queue event counts outside its scope. This qualification establishes receipt-confirmed activation under delivery load, probe-based redistribution with a 60% reduction in source subscriptions, and reconciled closure. The recorded delivery rates include workload depletion and startup, so the experiment reports redistribution rather than an isolated throughput effect.

I TTL-Based Reconnection Under Delivery Load

Time-to-live (TTL) reconnection is an existing alternative to client-directed connection replacement in the subscriber service. It was disabled in the preceding Oracle Cloud cluster experiments and enabled in the isolated recovery trials reported below. The deployment team reported that enabling age-based disconnection under event load caused widespread delivery interruptions and slow resubscription: delivery restarted on early-restored subscriptions while the

remaining subscriptions were still being attached. These operational reports provide context rather than a frequency or severity estimate. The matched-preload trial in Section I.5 measures expiry-related interruptions and ACK completion using server-correlated records.

I.1 Implemented Expiry and Recovery Paths

The server enables expiry only when the configured connection TTL is positive. For a connection established at time t_c^{open}, its nominal expiry is

$$t_c^{\text{exp}} = t_c^{\text{open}} + T_{\text{TTL}} + 60J_c, \qquad J_c \in \{1, \ldots, 60\},$$

where T_{TTL} is in seconds and J_c is the implementation's uniformly sampled integer jitter in minutes. A scheduled scan runs with a 60-second fixed delay. An expired connection is submitted to a cleanup executor, its server-side subscription associations are removed, and its socket is closed. Scheduling and cleanup add delay beyond t_c^{exp}; the scan interval is not a hard upper bound on closure latency. Several expired connections can be selected in one scan, although the random jitter reduces deliberate synchronization. The implementation does not imply that every client connection expires simultaneously.

The Python client reconnects the affected socket through the transport load balancer and resends the subscriptions retained for that socket. Recovery applies to the affected connection. It does not automatically restart the whole client pool. Each subscription retains its previous delivery state. Consequently, an active subscription requests active delivery immediately on resubscription; there is no whole-connection barrier that first restores every subscription and then releases their events together. Transport reconnection can therefore finish well before subscription restoration.

This sequence explains the two distinct concerns in the operational report. First, the expired socket cannot deliver while it is being replaced, and overlapping recovery intervals can interrupt a large fraction of the pool. Second, early-restored subscriptions can resume event delivery while other `SUBSCRIBE` operations are still outstanding. Delivery and restoration then overlap in the same service. The server's cleanup also traverses shared subscription state under a write lock. These operations can compete for shared resources. Their effects on throughput and latency depend on workload and cannot be inferred from connection age alone.

Table S9. Implemented TTL reconnection versus guarded DQL migration. This compares control mechanisms, not measured performance rankings.

Property	Existing TTL strategy	Guarded DQL migration
Trigger	Age, jitter and periodic scan	Observed pressure, Q-values and feasibility guards
Destination	Reconnect assigned by load balancer	Existing load-balancer-assigned probe evaluated before promotion
Replacement	Cleanup and close before reconnect	Target standby before source quiescence and drain
Subscription restart	Retained active subscriptions resume individually	Receipt-gated handoff and controlled target activation
In-flight work	No DQL drain barrier in TTL closure path	Source drain checked before retirement
Scope	Every connection eligible by expiry time	Bounded selected connection; other sockets retained

Table S9 identifies the intended advantage of probe-guided migration: destination evaluation and subscription handoff happen before the old socket is retired. TTL can also reach a newly added Pod, but selects by connection age rather than the destination's subscription or backlog pressure. DQL evaluates that pressure and controls source retirement; its handoff duration still includes source drain and target activation, so delivery can pause during the transfer.

I.2 Recovery Endpoints and Verification

Let d_c be the observed disconnect time, h_c the replacement `CONNECTED` time, and r_c the time when all retained subscriptions have successful replacement-session receipts. The two recovery endpoints are

$$T_c^{\text{conn}} = h_c - d_c, \qquad T_c^{\text{restore}} = r_c - d_c.$$

If recovery is incomplete at the next disconnect or the end of observation, we record only the elapsed waiting time. This is a right-censored observation, not a successful fast reconnect. Let s denote elapsed trial time in seconds, and let $I_c(s)$ equal one while connection c is disconnected or its retained bindings remain unrestored, and zero otherwise. With C_0 denoting the active connection pool at the start of a TTL trial, the affected fraction is

$$F(s) = \frac{1}{|C_0|} \sum_{c \in C_0} I_c(s).$$

The peak of F and its time integral distinguish a brief single-socket outage from widespread or prolonged recovery. Delivery-rate degradation on sockets that remain connected is measured separately; it is not counted as a physical disconnect. We also measure unique receipt-confirmed ACK rate, gaps between deliveries, pending subscription count, and receipt latency per subscription. These show the effects on both event delivery and subscription operations. Events delivered before r_c directly establish overlap between delivery and incomplete restoration, but overlap alone does not prove CPU or network saturation.

The recovery instrumentation requests receipts for reconnect subscriptions without serializing their dispatch. It preserves pending caller Futures, distinguishes successive sessions, and stores timestamps alongside the exact delivery ledger. A common instrumentation contract defines the measurements across comparison arms. Client disconnects are attributed to TTL only when matched to server expiry records; unrelated network failures remain separate.

The implementation-level checks exercise the disabled TTL branch, expired connection closure, and selection of expired rather than unexpired sockets in a common scan. Client checks cover receipt ownership, unchanged delivery states, interrupted recovery, and censored timing analysis. These checks validate implementation behavior. The live comparisons reported below measure expiry-related interruptions, receipt completion, and recovery timing using the native jitter. This separates observed behavior from an artificial simultaneous-disconnection scenario.

I.3 Recovery Observed Within the Existing Storage Limit

The isolated recovery trial retained the installed 400,000,000-byte Queue limit and consumed existing experimental events without a storage increase or new publication. Four clients attached 1,250 subscriptions each to one subscriber Pod, giving 5,000 receipt-confirmed standby subscriptions across 20 delivery connections and four probes. Each delivery connection held 250 subscriptions. Activation then released delivery with a bounded window of 16 outstanding requests per client. Native TTL used a configured value of 60 seconds with the original 1–60-minute jitter and scheduled scan; no simultaneous-disconnection rule was introduced.

A sealed client record captures a complete recovery of 250 retained subscriptions. The replacement socket reached `CONNECTED` in 0.018 seconds, whereas receipt-confirmed restoration took 104.174 seconds. During that restoration interval, 82 delivery attempts arrived on the replacement session. Thus, transport recovery did not mean that the subscription population was ready, and delivery demonstrably overlapped with restoration. The corresponding server expiry record was not retained. We therefore report this as recovery during a TTL-enabled trial, not as a disconnect independently confirmed to have been caused by TTL. A second client record does match an explicit server TTL-close request by session identity and records the subsequent disconnect; this partial run confirms that native expiry occurred, but it is not a completed performance trial.

The TTL-enabled execution ended during activation after a receipt timeout. One client sealed its evidence; the supervisor's earlier cleanup deadline interrupted the other three before sealing. These records are retained as diagnostics rather than treated as four successful repetitions. ACK errors also occurred on connections without a recorded disconnect, so they are not all assigned to TTL. The observed recovery sequence supports distinguishing socket readiness, subscription readiness and delivery activity; it does not by itself estimate a TTL-versus-DQL throughput effect.

After restoring TTL to its disabled state, a functional control used the same 5,000-subscription population and existing events. Its four verified client records contain 117,269 delivery attempts, 110,411 receipt-confirmed ACKs and 6,858 unresolved attempts. No active-connection interruption was recorded, but activation ended after an explicit upstream state-transition failure exhausted its retry budget; 2,929 bindings had confirmed active state in the last progress records. This distinguishes the expiry/recovery mechanism from an activation bottleneck that also occurs without TTL. Because the first execution had already consumed part of the workload, the two runs do not provide a matched-backlog throughput comparison. Both preserve the frozen model without additional training. The isolated Deployment was restored, and all thirteen shared-resource isolation checks passed.

I.4 Native TTL During Two Delivery Bursts

Before the preload trial, a separate live pair tested native TTL against a static TTL-disabled control on one isolated Oracle Cloud subscriber Pod. Both arms used 5,000 subscriptions, five clients, 20 occupied connections and five empty probes, with zero artificial client processing delay. Each started from an empty Queue and received two bursts of three accepted source events, giving 30,000 recorded delivery attempts per arm. Both used the same server image and 900-second post-activation observation. Native TTL retained its original jitter. The control replayed the accepted burst schedule: its first requests were 0.462 and 0.578 seconds later than the native relative targets of 26.182 and 319.207 seconds. Queue-statistics budget failures stopped other publication blocks before any event attempt; those failures were retained.

Table S10 shows the tail that a median alone would miss: native TTL's confirmed-ACK p95 was 4.27 times the control's, although its median was 1.968 seconds lower. Five interruptions matched server expiry records. Transport reconnection took 17–24 ms, whereas restoring all 250 bindings on each affected connection took 0.504–0.973 seconds. One recovery received 22 delivery callbacks before all binding receipts arrived. This directly demonstrates delivery overlapping resubscription within the observed short recovery windows; prolonged stalls and whole-pool expiry were not observed. All 369 unresolved attempts belonged to expired sessions; three clients reached the 120-second receipt-drain deadline. Missing receipts are not counted as lost Queue events. All ten sealed client bundles were recovered and verified. This single sequential pair measures TTL-related disruption and confirmed-receipt latency under static placement. It estimates neither repeated-run capacity nor a DQL-specific effect.

Table S10. Oracle Cloud two-burst TTL comparison, separate from the subsequent preload pair. Both arms use static placement on one Pod; TTL off is not a learned DQL intervention. Latency statistics include confirmed ACKs only.

Measurement	Native TTL	Static TTL off
Delivery attempts	30,000	30,000
Receipt-confirmed ACKs	29,631	30,000
Unresolved ACK receipts	369	0
Server-matched TTL interruptions	5	0
Confirmed ACK latency: median (s)	19.439	21.407
Confirmed ACK latency: 95th percentile (s)	238.392	55.826
Clients with receipt-complete shutdown	2 of 5	5 of 5

I.5 Matched-Preload Comparison in the Oracle Cloud Cluster

A subsequent isolated trial compared native TTL with the frozen DQL controller under the same initial queued workload. Each arm used the same 5,000 registered ET1 subscriptions: five clients held 1,000 subscriptions each on four occupied connections, with 250 subscriptions per connection and one empty probe per client. All 20 occupied connections initially terminated on one subscriber Pod. Three identical Pods became Ready before activation. Each Pod had a limit of three CPU cores and 6 GiB memory, and both arms used the same pinned server image. The installed Queue limit remained 400,000,000 bytes; shared subscriber resources were not changed.

All bindings were attached in standby before six source events were published through the authenticated ingest path. A pre-activation Queue snapshot verified 30,000 visible copies and zero in-flight copies in each arm. The recorded delivery ledger subsequently contained six attempts for every subscription in the same 5,000-subscription cohort. Native TTL retained the 60-second base, 1–60-minute random jitter and scheduled scan. The DQL arm disabled TTL and loaded unchanged online and target networks from the checkpoint with 9,985 training updates; exploration and further learning were disabled. Observation continued for 900 seconds after activation, with activation itself also recorded.

The original workload included a six-event refill at five minutes. Its first Queue-statistics budget check returned HTTP 429 before any event was attempted. Before the DQL arm started, the comparison was amended to use the six-event preload without refill. Three DQL preload budget checks also failed before publication; cooldown and fresh checks admitted the remaining events while all bindings stayed in standby. The initial copy counts therefore matched, but the setup proceeded at different rates. Queue occupancy was approximately 20.3 MB in both arms, not 400 MB. This sequential pair measures fixed-preload drainage. Sustained arrivals and subscriber saturation are outside that workload.

I.6 Receipt Completion and Connection Recovery

Table S11 reports the observed outcomes. Let D_{obs} be the number of recorded delivery attempts and A_{conf} the number with a confirmed ACK receipt. Receipt completion is $A_{\mathrm{conf}}/D_{\mathrm{obs}}$; the remainder is unresolved, not classified as event loss. Both arms recorded 30,000 attempts. Native TTL confirmed 26,831 ACKs (89.44%), compared with 30,000 (100%) in the TTL-disabled arm, a difference of 10.56 percentage points in this pair. The latter arm is labelled *guard-held DQL*: its controller ran, but telemetry guards held every decision at *stay*, as explained below.

All twelve native interruptions matched server expiry records by session identity and disconnect time. They affected eleven distinct occupied sockets, with at most four interrupted within a rolling minute. At most three sockets were concurrently unrestored, giving $\max_s F(s) = 3/20 = 0.15$ and $\int \sum_c I_c(s)\, ds = 10.983$ connection-seconds. Complete

Table S11. Measured Oracle Cloud outcomes for the matched 5,000-subscription preload. The DQL arm had TTL disabled and executed no migration. Latencies are conditional on confirmed ACKs; unresolved receipts are reported separately.

Measurement	Native TTL	Guard-held DQL
Delivery attempts	30,000	30,000
Receipt-confirmed ACKs	26,831	30,000
Unresolved ACK receipts	3,169	0
Receipt completion (%)	89.44	100.00
Server-matched TTL interruptions	12	0
Distinct occupied sockets interrupted	11 of 20	0 of 20
Peak simultaneously recovering sockets	3 of 20	0 of 20
Incomplete-restoration exposure (connection-s)	10.983	0
Confirmed ACK latency: median (s)	69.862	69.836
Confirmed ACK latency: 95th percentile (s)	251.546	249.874
Executed DQL migrations	Not applicable	0

binding restoration took 0.492–1.835 seconds. These observations show overlapping recovery periods affecting part of the pool, rather than a whole-pool outage. All 3,169 unresolved native attempts belonged to sessions with matched expiries, but a missing ACK receipt does not reveal whether the underlying Queue deletion succeeded.

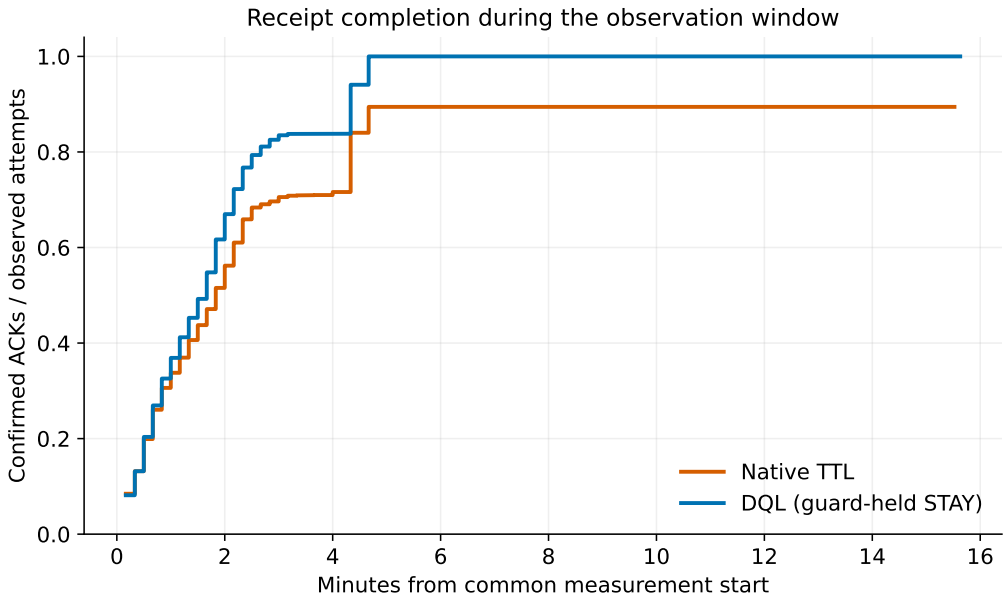


Fig. S9. Receipt completion in the Oracle Cloud matched-preload pair. Each curve accumulates confirmed ACKs in ten-second bins and divides by all 30,000 observed attempts in its arm. Unresolved attempts remain in the denominator. DQL denotes the guard-held, TTL-disabled execution, not a completed migration comparison.

Figure S9 retains the unresolved attempts in the final level rather than rescaling each curve using only successful attempts. Time starts when all five client observation windows overlap within each arm. Those common windows lasted 931.347 and 937.743 seconds, respectively. Their mean confirmed-ACK rates were 28.809 and 31.992/s, including

idle time; these are whole-window completion rates, not estimates of maximum Pod bandwidth. Four native clients reached the 120-second final ACK-drain deadline. We retain their incomplete drains and the coordinator's nonzero exit status as outcomes. All ten client evidence bundles were sealed and verified; the DQL coordinator completed cleanly.

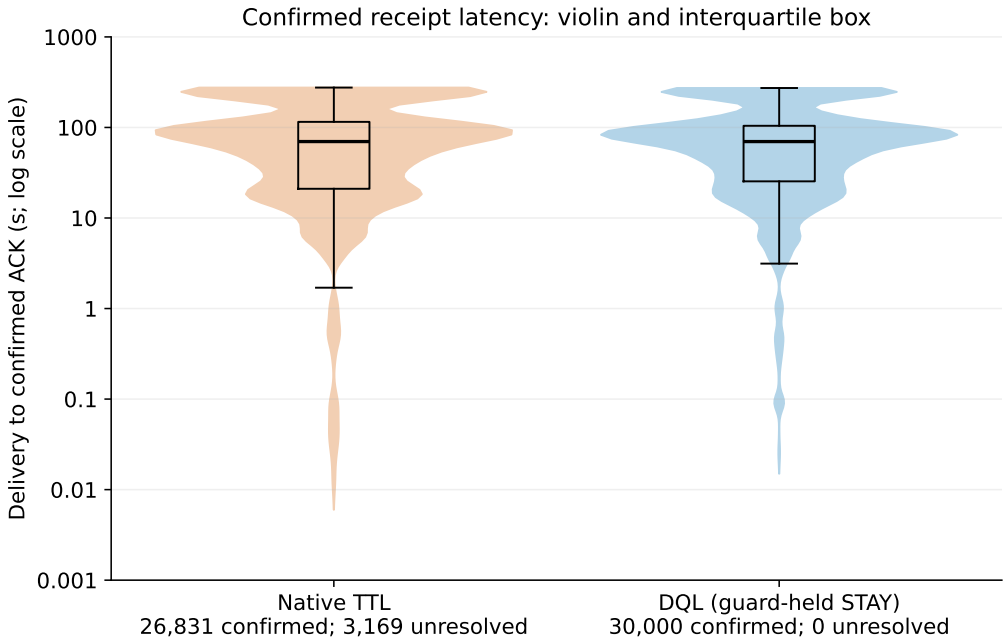


Fig. S10. Confirmed delivery-to-ACK latency in the Oracle Cloud preload pair. Violin density is estimated on $\log_{10}$ latency; boxes show its median and interquartile range, with 1.5-IQR whiskers. Outlier markers are suppressed, not removed from the observations. Counts exclude unresolved receipts.

Figure S10 separates receipt completion from conditional latency. The confirmed medians differ by only 0.026 seconds, and the 95th percentiles are also close. Native unresolved receipts are absent from this latency distribution but remain explicit in Table S11. Violin width describes within-run latency variation. The five clients and 30,000 attempts belong to one pair rather than independent trials, so between-run uncertainty is not estimated. Unlike the earlier 104.174-second diagnostic, this pair recorded no new delivery callbacks during subscription-restoration windows. It captures a different recovery condition from resubscription under concurrent delivery.

I.7 Telemetry Freshness and Action Eligibility

The DQL trace contains 450 decision intervals, all held at *stay* by `invalid-backlog-telemetry`. No migration executed, and all 5,000 bindings remained on the source Pod despite four Pods being Ready. Statistics collection paced individual subscription requests at 40 ms for a 5,000-subscription sweep. Even before network delay or retries, the sweep required at least $5{,}000 \times 0.040 = 200$ seconds, exceeding the 180-second oldest-sample deadline. The minimum coverage across the Pod and occupied-connection components ranged from 0.560 to 0.808. Only ten observations met every 80% coverage threshold; their oldest constituent samples were 183.127–185.974 seconds old and failed freshness. A recently received telemetry frame can therefore contain stale Queue observations. Queue-statistics rate limits further constrained coverage.

This finding separates two operational properties. Native expiry produced measurable interruptions and receipt uncertainty; the TTL-disabled, guard-held arm preserved its occupied sockets and completed all observed ACKs. Freshness checks held migration inactive, so the measured benefit concerns connection retention and receipt completion, rather than throughput from added Pod capacity. The checkpoint and guards stayed fixed throughout the trial. Cleanup verified empty Queue and dead-letter Queue inventories, restored isolated TTL to zero and one Ready Pod, and passed all thirteen shared-resource isolation checks.

J Additional Scale-Out and Timing-Control Results

The workload, service model, primary comparison, and backlog trajectories appear in the main paper. This section retains the capacity-activation plots, distributions, and exploratory timing controls for the same experiment.

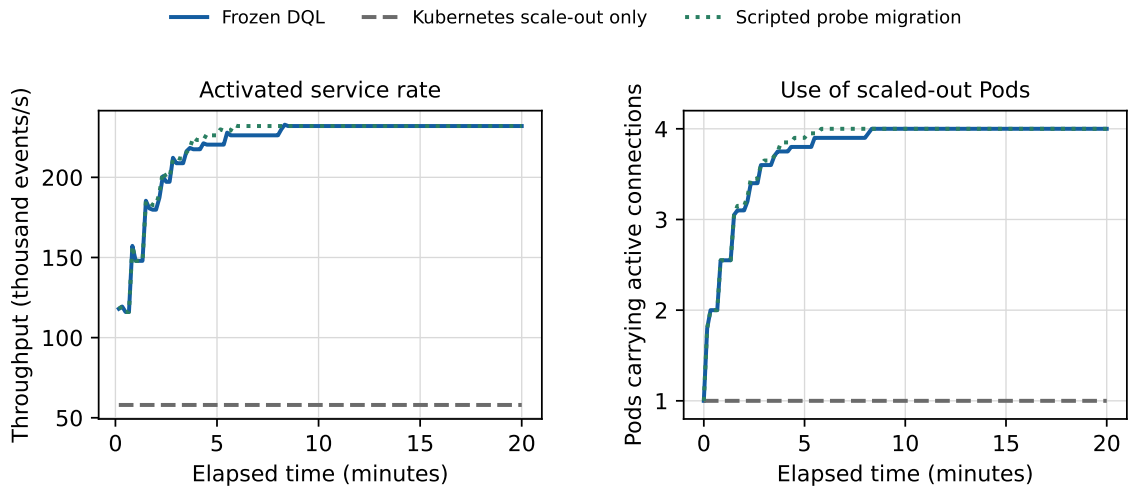


Fig. S11. Use of the added subscriber capacity. Left: mean delivered events per second over each measured interval. Right: mean number of pods carrying active client connections. The ten delivery connections are retained as a bounded pool while probe promotion replaces their endpoint assignments.

Figure S11 provides the mechanism evidence: the throughput increase occurs as more pods acquire subscription-bearing connections. Added pod capacity and client-directed reassociation jointly produce the gain. In the model's saturated regime, a single pod supplies approximately 58,000 events/s; four utilized pods supply approximately 232,000 events/s. The mean over the full recovery period is lower because probe discovery and cooldown take up part of the observation window. The maximum relative difference in queue accounting is 1.45×10^{-13} across the 60 episodes. The simulator represents reassociation as a discrete action; network delays and the live receipt protocol are outside this timing model.

Figure S12 makes the seed-to-seed variation behind the recovery averages visible. Each AUC observation is the episode-level quantity in Eq. (29 of the main paper); each throughput observation is the episode's delivered-event total divided by its 1,200-second duration. The violin uses a Gaussian kernel with Scott's bandwidth, $h = s_{\text{sample}} n_{\text{ep}}^{-1/5}$ for sample standard deviation s_{sample} and $n_{\text{ep}} = 20$, and is drawn only between the smallest and largest observations. Its width is descriptive, not a confidence interval. The scale-out-only outcomes are constant because that policy never

changes its initial placement. The adaptive distributions reflect variation in probe discovery and migration timing. Their separation from the baseline agrees with the paired AUC result above. The overlapping DQL and scripted distributions describe similar recovery performance for these two policies, without establishing DQL superiority over the guarded scripted control.

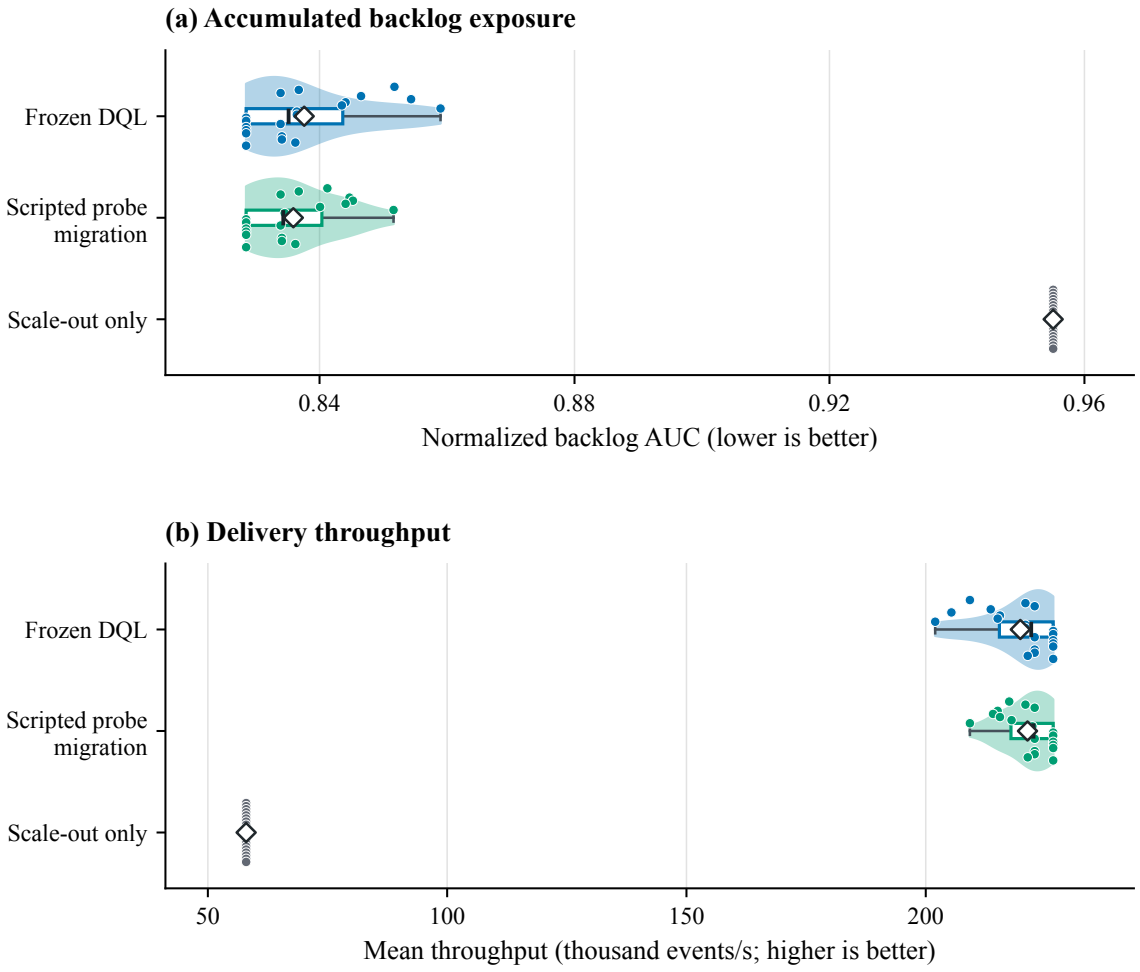


Fig. S12. Scale-out recovery distributions over 20 paired seeds per policy. Violin width shows a kernel-density estimate within the observed range; boxes show quartiles and medians, diamonds show means, and points show all episodes. Whiskers use the 1.5-interquartile-range rule. The constant scale-out-only baseline is shown without an artificial density. Lower normalized backlog AUC and higher throughput are preferred.

Independent ad-hoc migration. An exploratory extension retains all sixty original observations and adds eighty episodes: four pressure-unaware schedules evaluated on the same twenty paired seeds. Each attempt chooses a connection and target Pod uniformly, without consulting backlog, pressure, or the learned policy. A same-Pod draw remains a no-op. The schedules use fixed periods of 40 or 60 seconds, or a 60-second nominal period with ±30% or ±40%

uniform timing jitter. All schedules first act at interval zero. The nominal jitter ranges are 42–78 and 36–84 seconds; rounding upward to the ten-second decision clock gives executable ranges of 50–80 and 40–90 seconds, respectively. These percentages modify the schedule, not the measured throughput. Rounding also changes the average attempt frequency, so jitter is not isolated from timing. In the following comparison, *Scripted* denotes the independent 60-second schedule with ±30% timing jitter; *guarded-greedy migration* denotes the original pressure-aware positive control. These are distinct non-learning policies.

Table S12. Independent ad-hoc migration sensitivity in the scale-out simulator. Means over twenty paired seeds per policy; timing jitter changes the schedule, not measured values.

Policy	Events/s	Backlog AUC	Migrations	Final Pods
Frozen DQL	219,762	0.83759	7.75	4.00
Guarded-greedy migration	221,260	0.83589	7.75	4.00
Ad-hoc: fixed 40 s	205,295	0.84997	22.85	3.70
Ad-hoc: fixed 60 s	197,636	0.85906	15.40	3.90
Scripted	195,737	0.86064	14.70	3.85
Ad-hoc: 60 s, ±40%	195,640	0.86053	14.65	3.85
Scale-out only	58,000	0.95510	0.00	1.00

Table S12 shows mean DQL throughput of 219,762 events/s, compared with 195,737 for the ±30% schedule and 195,640 for the ±40% schedule. The relative gains are 12.27% and 12.33%, with pointwise paired 95% bootstrap intervals of [9.38%, 15.55%] and [9.31%, 15.81%], respectively. Each interval uses 10,000 resamples of complete seed pairs and the ratio of resampled mean rates. DQL uses 7.75 migrations per episode, compared with 14.70 and 14.65 for the two jittered schedules. The fixed 40-second and 60-second schedules yield gains of 7.05% and 11.20%, so the comparison covers both fixed and jittered schedules. This exploratory sensitivity analysis reuses the scenario seeds. Its scope is simulated migration scheduling, separate from independent holdout testing and live connection-TTL measurements.

Figure S13 shows the variation hidden by the averages. Random reassignment can revisit an already occupied Pod or move work away from a useful placement, whereas the guarded DQL controller admits pressure-improving candidates. The distributions and migration counts compare the complete controllers, including their admission and endpoint-selection rules. The nearby guarded-greedy distribution shows that the result compares complete policies, including their guards, rather than isolating the effect of learned action selection.

Figure S14 summarizes the same stressed workload, rather than importing the mixed-workload radar's averages. Its throughput coordinate divides each episode's mean rate by the four Pods' combined nominal capacity, $4 \times 100{,}000 = 400{,}000$ events/s. The other coordinates are the fraction of initial backlog removed, the final active-Pod count divided by four, and $10/(10 + M^{\mathrm{ep}})$ for an episode with M^{ep} migrations. The last coordinate decreases with intervention count; it is not a handoff-latency measurement. Each plotted point averages these per-episode coordinates across all twenty seeds. No observed-best or policy-specific denominator is used. The ±30% ad-hoc schedule appears alongside the original three controls; all timing variants remain in Table S12.

The DQL throughput radius is 12.3% larger than the Scripted radius and 3.79 times the scale-out-only radius because all use the same capacity denominator. The guarded-greedy comparator has a nearly overlapping profile and slightly higher mean throughput. The result establishes the benefit of reaching added capacity, with similar performance from a pressure-aware non-learning control. Backlog removal and throughput are linked because the workload has no new

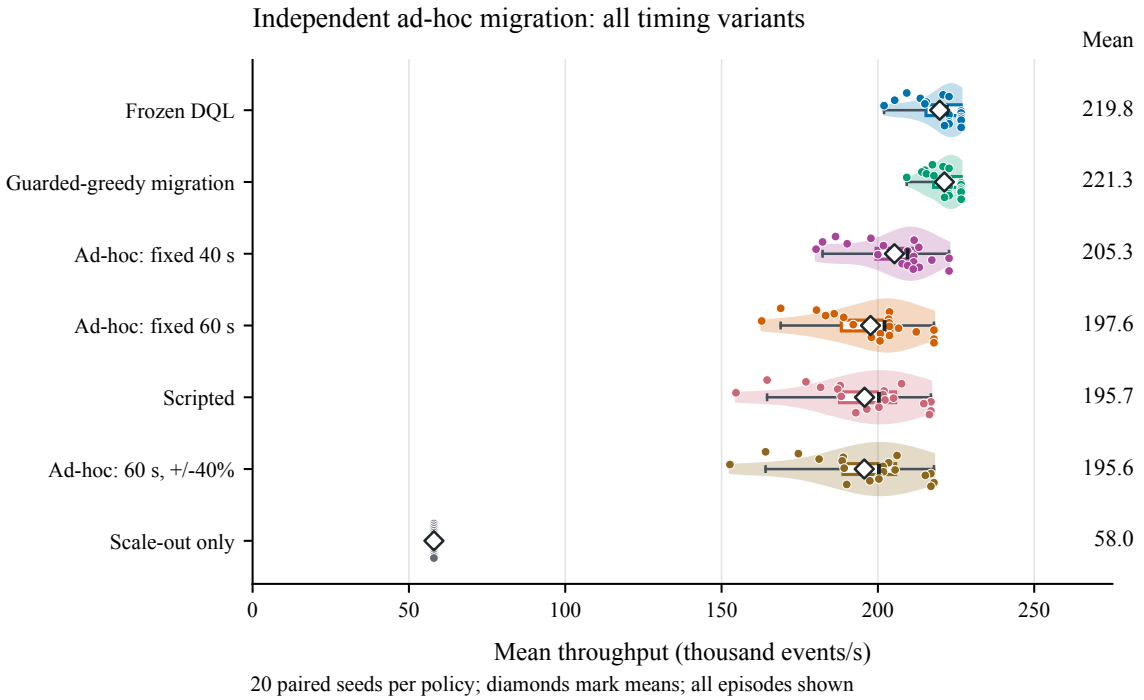


Fig. S13. Throughput under independent ad-hoc migration schedules. All twenty episodes per policy are shown. Violin density uses Scott's bandwidth within the observed range; boxes show quartiles and medians, whiskers use the 1.5-interquartile-range rule, and diamonds mark means. The constant scale-out-only control has no artificial density. All seven policies are retained, including the original guarded-greedy control.

arrivals during drainage. They are not independent confirmations. The polygon's area is not a performance measure. These measurements remain distinct from the Oracle Cloud expiry and receipt-completion results in Section I.5.

K Full Supporting Simulator Studies

Study designs and explanatory passages are retained with their original results. Selected definitions and results also appear in the main paper so that its core comparison can be read independently.

K.1 Supporting Study Design

The independent v3 holdout compares a frozen DQL policy with deterministic controls. The adverse-probe ablation holds that checkpoint fixed and varies the promotion guard, using separate 30-seed inference and five-seed scale cohorts. Expanded v4–v6 studies evaluate action coverage and contract changes; they are development diagnostics, not additional holdout performance results. Table S13 identifies these version boundaries. Appendix L retains their detailed outcomes.

The shared frozen deployment file has SHA-256 prefix `3e68371d848eca85`; the reviewer evidence package retains the full hash and weights. A missing contract field identifies the legacy checkpoint. The current client permits it only for a compatible restricted action profile; full-action deployment requires the versioned contract.

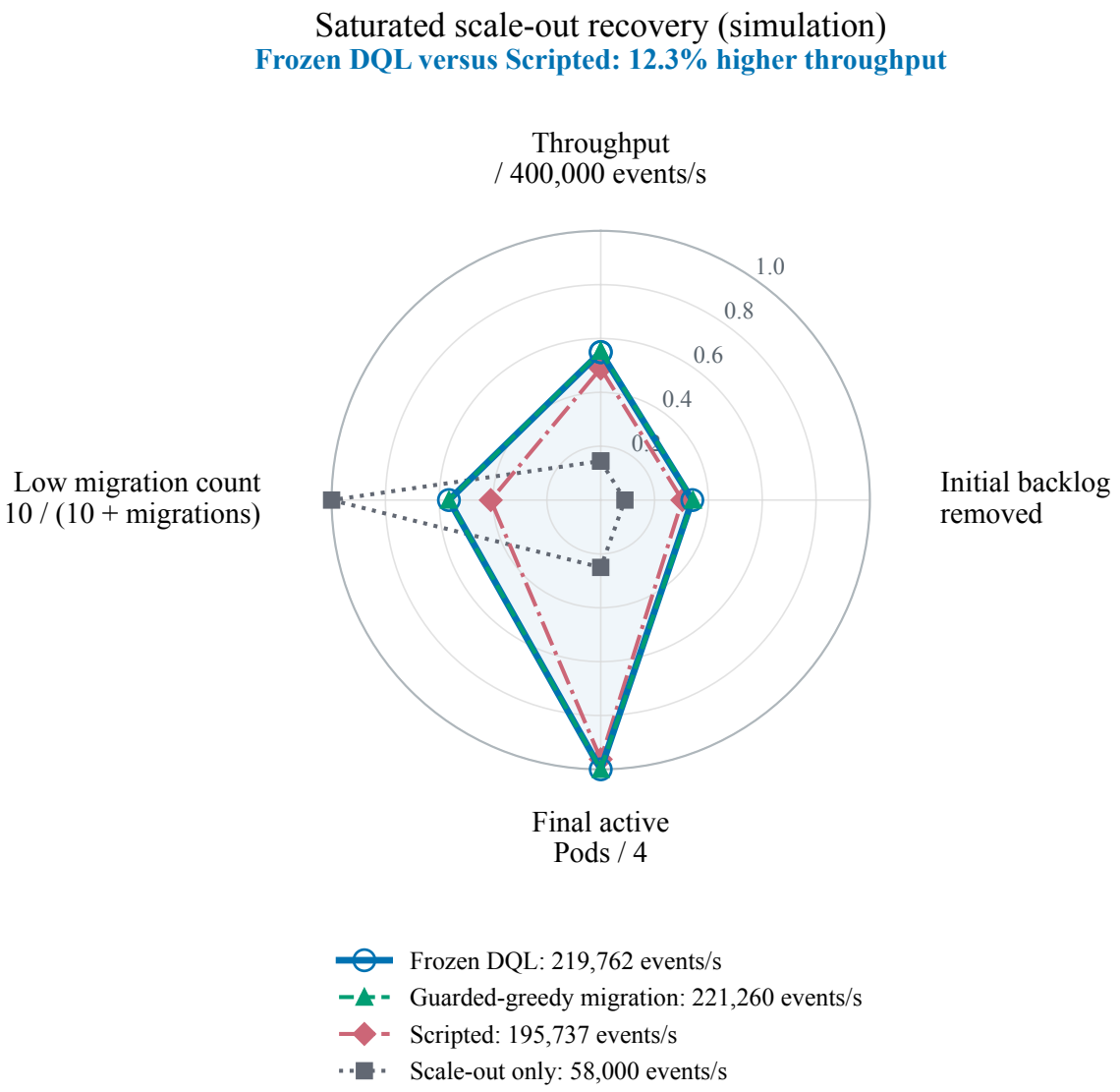


Fig. S14. Stress-scenario radar from all twenty paired simulator seeds per displayed policy. **Frozen DQL versus Scripted: 12.3% higher throughput**, with measured mean rates of 219,762 and 195,737 events/s, respectively. The guarded-greedy and scale-out-only controls remain visible at 221,260 and 58,000 events/s. All coordinates use fixed zero-to-one scales, with outward preferred; migration count exposes the intervention tradeoff. This is a descriptive profile, not an additional objective or statistical test.

K.2 Research Questions and Baselines

The evaluation addresses four questions. **RQ1** asks whether a frozen DQL policy improves the multi-objective return and connection distribution relative to a deterministic, two-dimensional heuristic. **RQ2** asks how the policy trades off the individual operating measurements alongside the scalar reward. **RQ3** asks whether independently trained candidates use every allowed action in its intended test scenario while respecting the fault guards. **RQ4** asks whether

Table S13. Checkpoint, action-contract, and execution scope. A policy trained under one action meaning is not evidence for another meaning.

Study	Policy / action contract	Scope of the evidence
v3 holdout and adverse-probe ablation	Frozen seed 101; legacy open-only scale-up	Reported rebalancing and multi-objective results; no cap-restoring claim
20,000-subscription scale-out	Same frozen checkpoint; fixed ten-connection pool	Endpoint migration; workload does not admit scaling actions
October loaded cluster trial	Same checkpoint; explicit rebalancing-only profile	Scaling actions disabled; three selective live migrations
v4–v5 development	Legacy scale-up; v5 corrects the scale-down guard	Action-coverage diagnostics; no selected holdout policy
v6 development	Versioned cap-restoring scale-up	Scripted action preflight and learner gates; no selected holdout policy

rejecting an adverse probe promotion reduces the amount of queued work over time compared with overriding that guard, blind promotion, static placement, and the guarded heuristic.

The v3 holdout compares three policies. *Static* retains the initial transport assignment. *Heuristic* applies the threshold-based subscription-count and backlog rules without value learning. *Frozen DQL* uses the checkpoint selected on the fixed development-validation bank, sets $\epsilon = 0$, disables training, and invokes the same action masks and execution guards as the deployed client. The heuristic is the primary baseline because it has the same rebalancing authority as DQL; the static policy is the no-rebalancing control for the initial connection assignment.

During each episode, the policy receives only client-deployable observations: local subscription queues, active-connection telemetry, interval throughput counters, and one retained probe candidate. Cluster-wide pod distributions are calculated by the simulator only after an action for that interval has been chosen. They are evaluation outputs and are not inserted into the policy state. This maintains the partial-observability boundary formalized in Section 4 of the main paper.

K.3 Predeclared Protocol

Patterson et al. identify hyperparameter selection, baseline construction, and uncertainty reporting as central issues in empirical RL evaluation [29]. We therefore keep development selection separate from the locked holdout and report component metrics alongside return. Testing adaptation under changed execution conditions is also distinct from measuring performance in the training setting, as demonstrated by Biagiola and Tonella's plasticity analysis [3]. The operating and fault scenarios below test this distinction without treating successful training as evidence of universal adaptation.

Table S14 summarizes the evidence-generating runs. In v3, three candidate networks were trained for exactly 10,000 transitions under seeds 101, 211, and 307. Candidate selection used ten fixed validation seeds and required subscription-count rebalancing ($a^{\mathrm{reb}N}$), backlog rebalancing ($a^{\mathrm{reb}G}$), and full connection migration (a^{mig}) in at least 80% of admissible episodes in their intended scenarios. Candidate 0 (seed 101) passed this gate and was frozen before the holdout seed bank was read.

The independent v3 holdout contains ten previously unused seeds for each of four scenarios and 500 decision intervals per episode. The connection-count skew scenario uses 1,000 subscriptions; the tiered-backlog scenario uses 1,200 subscriptions; and the client-slowdown and pod-pressure-skew scenarios each use 1,600 subscriptions.

Table S14. Locked simulator study design. "Not opened" means that no performance episode was run on the reserved holdout seed bank.

Study stage	Training budget	Fixed evaluation	Selection requirement	Holdout status
v3 development	3×10,000 transitions	10 validation seeds; 4 scenarios	$a^{\mathrm{reb}N}$, $a^{\mathrm{reb}G}$, and a^{mig} each at $\geq$ 80% episode coverage	Not opened
v3 holdout	Frozen seed 101; no updates	10 new seeds; 4 scenarios; 3 policies; 120 episodes	No selection on holdout	Completed
v4 uniform replay	5×20,000 transitions	10 validation seeds; 7 operating and 4 fault scenarios	All five intervention actions at $\geq$ 80%, plus all safety guards	Not opened
v4 stratified replay	5×20,000 transitions	Same fixed v4 validation bank	Same all-action and safety gate	Not opened
v5 corrected guard	5×20,000 transitions	10 fresh validation seeds; same 11 scenarios	Same all-action and safety gate	Not opened
v6 scale-up preflight	No learner training	10 paired seeds; 500 intervals	Target metric and reward improve for every seed, plus safety gates	Not defined
v6 learner factorial	$4 \times 5 \times 20{,}000$ transitions	10 fresh validation seeds; same 11 scenarios	At least 4/5 candidates pass every action and safety gate	Not defined
Adverse-probe ablation	Frozen v3 checkpoint; no updates	30 paired inference seeds and 5 paired 20k scale seeds; 500 intervals	Lower paired backlog AUC for guarded DQL than forced promotion	Completed

Every scenario has four modeled pods, four initial persistent connections, and a maximum of six connections. The tiered-backlog scenario assigns 25% of its queues to each initial backlog level 100,000, 50,000, 5,000, and zero. The client-slowdown scenario reduces acknowledgement capacity to 35% of its nominal value between intervals 125 and 375. The other two scenarios isolate initial subscription-count skew and the placement of three of four connections on one pod, respectively.

For policy π, paired episode j, and metric v, let $Y_{\pi,j,v}$ denote the episode mean. We define a direction-adjusted paired difference

$$D_{j,v} = d_v \left(Y_{\mathrm{DQL},j,v} - Y_{\mathrm{base},j,v} \right), \qquad d_v = \begin{cases} +1, & \text{higher is better,} \\ -1, & \text{lower is better.} \end{cases} \tag{S23}$$

Thus, a positive $D_{j,v}$ always favors DQL. The relative benefit reported below is

$$\mathrm{RB}_v = 100 \, \frac{\overline{D}_v}{\left| \overline{Y}_{\mathrm{base},v} \right|}. \tag{S24}$$

Overbars in these two equations denote means across paired episodes. The metric index v is distinct from the client-population size m. We resample the 40 paired differences 10,000 times to obtain percentile 95% confidence intervals. Two-sided paired sign-flip tests are exact for the scenario-level sets of ten pairs and use 100,000 Monte Carlo sign vectors for the pooled set of 40 pairs. Pooled p-values are adjusted by the Holm procedure across the complete family of 18 metric-by-baseline tests. The scenario box plots in Figure S16 describe quartiles and individual episode outcomes; the paired randomization tests provide the statistical comparison.

We evaluate DQL as a sequential controller through return, operational outcomes, paired effect sizes, and action coverage. Classifier accuracy and ROC–AUC apply to labelled prediction tasks. Applying them here would mainly

measure agreement with a heuristic that supplied the correct-action labels, rather than the controller's operating performance.

K.4 Training Diagnostics and Frozen-Policy Selection

Figure S15 shows the archived v3 development traces. The upper panel reports the cumulative mean interval reward $J^{-1}\sum_{\ell=1}^{J} R_{\ell}^{\text{train}}$, where J counts logged training transitions and R_{ℓ}^{train} is the recorded reward of transition ℓ. This is a mean, not accumulated reward, so its scale remains comparable across checkpoints. Candidate 0 recovered from the early high-exploration regime and ended with the highest development mean reward. The other candidates follow similar but distinct trajectories, which shows that the random seed has a clear effect on the learned values.

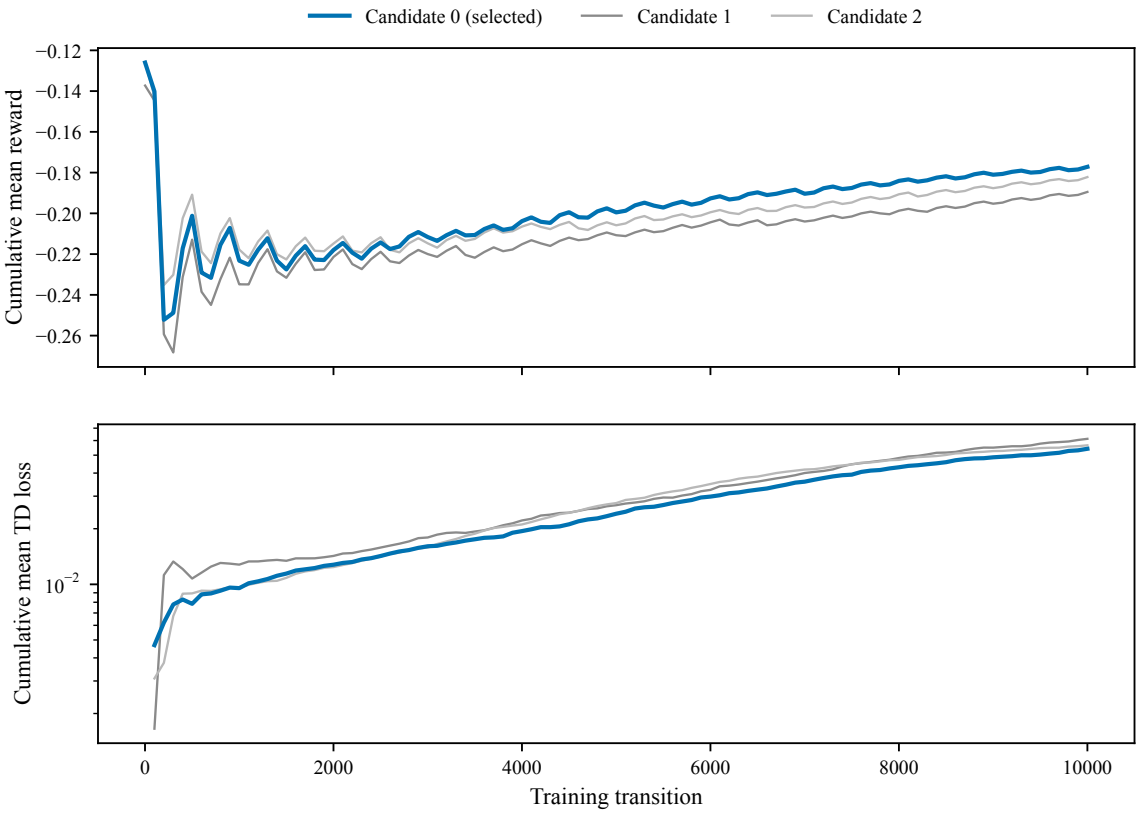


Fig. S15. Development traces for the three v3 candidates. Candidate 0 was selected by the predeclared validation rule and then frozen. The lower panel uses a logarithmic axis for the cumulative mean squared temporal-difference (TD) loss.

The cumulative mean TD loss rises for all three candidates after the early updates, with all archived values finite and bounded. This is a training diagnostic, not a convergence result. The target changes with the online network and the states visited during training, so TD loss measures target fit rather than model accuracy. Candidate selection used frozen-policy validation and action coverage. The independent holdout supplies the comparative performance results below.

K.5 Independent Holdout Results

Figure S16 shows that outcomes differ across scenarios. Frozen DQL has a higher return than the heuristic for connection-count skew, tiered backlog, and client slowdown. DQL and the heuristic are indistinguishable in the pod-pressure scenario because both select the same effective guarded response. DQL also matches the static policy under client slowdown: with client acknowledgement capacity as the bottleneck, additional connection movement leaves the service-rate bottleneck unchanged. These control cases identify when the policies provide equivalent outcomes.

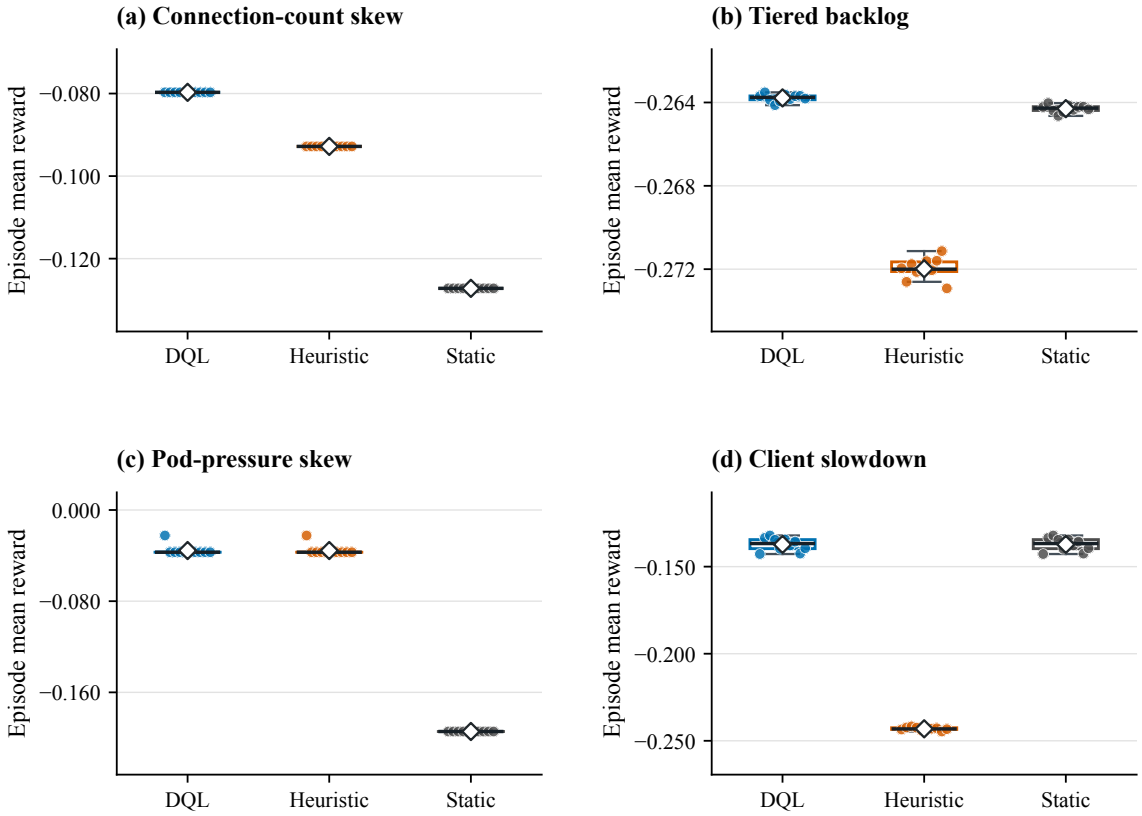


Fig. S16. Holdout reward distributions for frozen DQL, heuristic, and static policies. Each panel contains ten paired seeds per policy. Boxes span the first and third quartiles, the central line is the median, whiskers extend to observations within 1.5 interquartile ranges, and diamonds mark means. Every episode is shown as a point, including observations beyond whiskers. Panels use separate vertical scales; higher reward is better.

The boxes in Figure S16 describe the distribution of episode means, not the distribution of individual decision intervals. Horizontal point offsets separate the ten observations without changing their rewards. Collapsed boxes in the connection-count scenario reflect identical outcomes, not missing data. Separate panel scales make the smaller spread in the tiered-backlog case visible. The paired estimates below provide the statistical comparison, not the visual height of a box.

Across all 40 paired episodes, frozen DQL improves mean reward over the heuristic from −0.1609 to −0.1291. The mean paired difference is 0.0318 (bootstrap 95% CI [0.0191, 0.0455]), equivalent to a 19.8% relative benefit under Eq. (S24);

the paired standardized effect is $d_z = 0.729$, and the Holm-adjusted sign-flip p-value is below 0.001. Table S15 and Figure S17 show that this overall gain comes with tradeoffs.

Table S15. Frozen DQL versus the heuristic on the independent v3 holdout. Positive benefit favors DQL. CIs are paired-bootstrap intervals transformed by Eq. (S24); p_H is Holm-adjusted over 18 pooled metric-by-baseline tests.

Metric	DQL	Heuristic	Benefit [95% CI] (%)	p_H
Mean reward	−0.1291	−0.1609	19.8 [11.9, 28.3]	< .001
Backlog ratio	0.1749	0.2002	12.7 [−1.2, 27.1]	.095
Conn.-subscription variance	574.7	5768.7	90.0 [61.0, 120.8]	< .001
Conn.-backlog variance	2.280×10^{13}	1.568×10^{13}	−45.4 [−72.5, −22.6]	.022
Subscription imbalance	0.0898	0.4343	79.3 [59.1, 99.5]	< .001
Backlog imbalance	0.2194	0.1638	−34.0 [−56.4, −13.6]	.022
Combined conn. imbalance	0.1546	0.2991	48.3 [33.8, 63.8]	< .001
Pod pressure	0.1878	0.2051	8.4 [3.6, 13.6]	.022
Throughput ratio	0.2840	0.2973	−4.5 [−7.1, −2.2]	.022

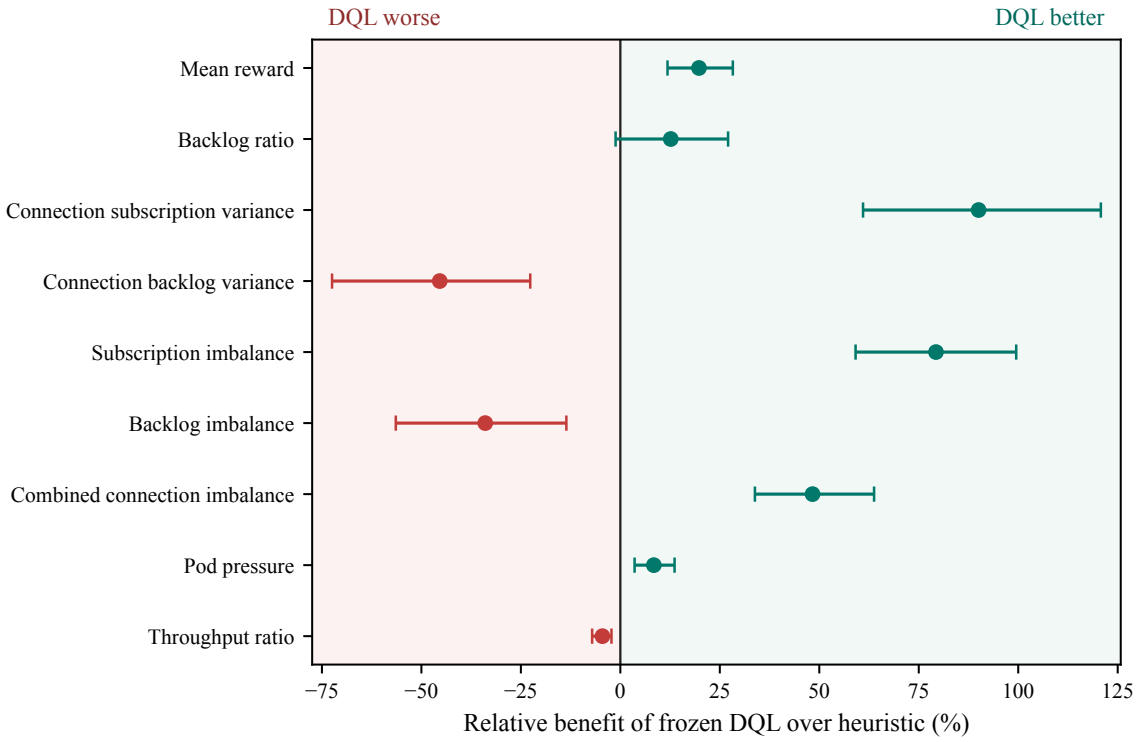


Fig. S17. Direction-adjusted relative benefit of frozen DQL over the heuristic across 40 paired holdout episodes. Points are mean effects and whiskers are paired-bootstrap 95% confidence intervals. Values right of zero favor DQL; values left of zero favor the heuristic.

The largest benefit is a 90.0% reduction in connection-level subscription variance, consistent with the design objective of moving subscriptions away from overloaded persistent connections. Subscription imbalance falls by 79.3%, combined

connection imbalance by 48.3%, and mean pod pressure by 8.4%. Mean backlog ratio is 12.7% lower, but its interval crosses zero and the adjusted test is not significant ($p_H = .095$). Conversely, DQL has 45.4% higher connection-backlog variance, 34.0% higher normalized backlog imbalance, and 4.5% lower throughput than the heuristic. The scalar reward still favors DQL because its configured penalties place substantial weight on subscription pressure and combined imbalance. The holdout therefore identifies a multi-objective advantage for DQL and a queue-drainage advantage for the heuristic.

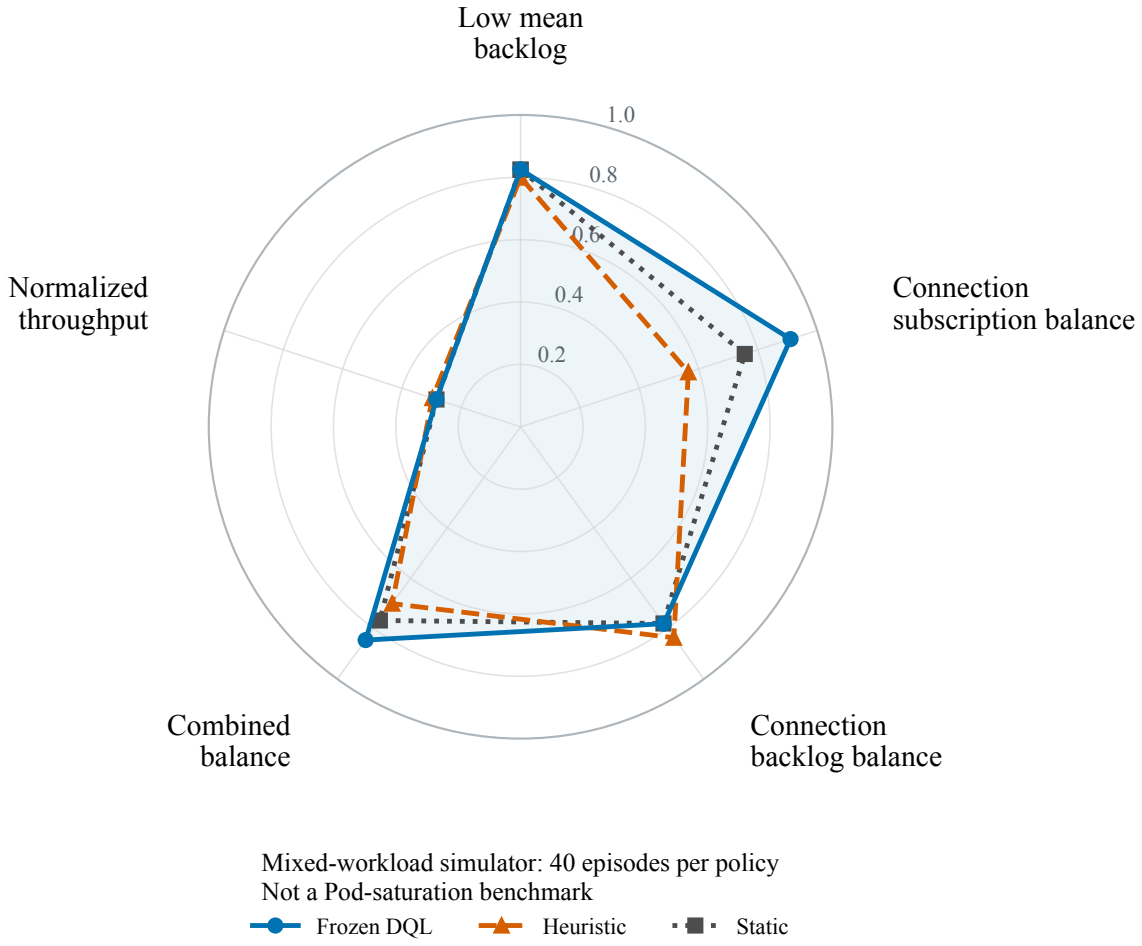


Fig. S18. Mixed-workload simulator profiles, not a Pod-saturation benchmark, across 40 paired holdout episodes per policy. Subscription balance is measured across client connections, not across all Pods. All axes use a fixed zero-to-one scale with larger values preferred. Backlog and imbalance costs are displayed as one minus their mean normalized value; throughput retains its recorded normalized value. The profiles are descriptive, not an additional weighted score or significance test.

Figure S18 summarizes these tradeoffs without mixing raw variances with event rates. For policy π and a normalized cost metric x_v, the plotted coordinate is $1 - \bar{x}_{\pi,v}$; the throughput coordinate is $\bar{x}_{\pi,\text{throughput}}$. Here each mean covers the same 40 episodes, with ten episodes from each scenario. No observed minimum or maximum is used to stretch

an axis. DQL's stronger subscription and combined balance coexist with the heuristic's stronger backlog balance and throughput. Polygon area has no statistical interpretation, and the combined-balance axis is derived from the two imbalance components rather than being an independent outcome. Uncertainty and paired comparisons remain those in Table S15 and Figure S17.

This pooled profile characterizes the mixed workloads; Pod saturation is evaluated separately. The connection-count-skew workload has no arrivals or initial backlog and therefore contributes zero throughput for all policies. In the pod-pressure workload, all policies serve the offered traffic with essentially zero residual backlog; their throughput is therefore equal despite different Pod pressure. The client-slowdown workload is limited by client acknowledgement capacity. Count balance and recovery of overloaded-Pod bandwidth are therefore separate endpoints. The saturated scale-out comparison is reported separately in Figures 3 of the main paper and S12.

The distinction is operational, not merely graphical. Equal subscription counts can leave every persistent socket on the same saturated Pod, and moving idle subscriptions does not redistribute queued work. Conversely, moving backlogged subscriptions to a Pod with spare service capacity can raise aggregate throughput, provided client ACK capacity and shared Queue or network limits do not become the bottleneck. In the scale-out experiment, the initial connection counts and backlogs are already balanced ($V_u^N(0) = V_u^G(0) = 0$), yet all ten sockets terminate on one Pod. Migration across Pods raises measured simulator throughput from 58,000 to 219,762 events/s (3.79 times). Thus, the throughput-recovery evidence is the stressed placement comparison in Section 7.1 of the main paper. Connection-balance scores describe a different property. The Oracle Cloud preload pair in Section I.5 measures expiry and ACK completion with the migration guard held at stay.

The comparison with static control isolates the benefit of rebalancing under concentrated load. In the 1,600-subscription pod-pressure scenario, three of four active connections initially terminate on the same pod. Static placement therefore retains mean pod pressure 1.0125, whereas DQL promotes a guarded probe target and reduces it to 0.2176, a 78.5% reduction. The corresponding paired reward improvement is 0.1588 (95% bootstrap CI $[0.1573, 0.1617]$). In the connection-count-skew scenario, DQL reduces mean connection subscription-count variance from 6,250.0 to 2,298.7 (63.2%) and improves mean reward by 0.0475. Across all four scenarios, pooled mean reward improves by 0.0517 relative to static placement. The scenario plot also identifies cases where DQL and static placement are equal. The paired analysis assigns each measured gain to its workload, keeping the concentrated-load and tiered-backlog results separate.

K.6 Action-Contract Validation

The v3 policy passed the declared subscription-count, backlog-rebalancing, and migration coverage gates. The expanded v4–v6 studies tested all five interventions and fault guards. None passed every learned-action coverage gate, although the v6 candidates all passed the safety scenarios. These completed development studies do not supply additional holdout performance estimates. Appendix L retains their full results, including the scripted test of the corrected scale-up contract.

K.7 Adverse-Probe Promotion Ablation

The retained probe is a candidate endpoint, not an instruction to migrate. To isolate this distinction, we registered a paired mechanism experiment before the confirmatory run. In every episode, the probe was set to reach the pod with the highest initial pressure. The deployment mask evaluated the concrete source–target pair and rejected a^{mig} whenever its projected pressure improvement was below 0.01. The primary ablation then bypassed that mask for exactly one interval, moved the highest-backlog non-target connection to the adverse pod, and returned control to the same frozen

DQL checkpoint. Thus, *Frozen DQL* and *DQL + forced promotion* differ only by that initial override and its subsequent state consequences. *Blind promotion* makes the same move and then stays; *Static* never changes the initial placement; and the guarded heuristic uses the same feasibility mask as DQL.

For total backlog $B(t)$ in the evaluated subscription population over T decision intervals, the primary endpoint is the normalized trapezoidal backlog area

$$\mathrm{AUC}_B = \frac{1}{T}\sum_{t=1}^{T}\frac{B(t-1)+B(t)}{2B(0)}. \tag{S25}$$

It measures how much work remains queued over time. Lower values indicate earlier drainage. The statistical unit is the environment seed, not an interval. The primary cohort uses 30 previously unused paired seeds and 1,200 subscriptions.

Each episode runs for 500 intervals. Paired mean differences use 10,000 bootstrap resamples and one-sided sign randomization with 100,000 sign vectors; Holm adjustment covers the four named baselines. The action mask rejected the adverse migration in all 30 primary and all five scale-confirmation environments.

In the primary cohort, guarded DQL attains mean $\mathrm{AUC}_B = 0.66046$, compared with 0.68938 after the forced promotion. The paired difference is -0.028918 (95% CI $[-0.039444, -0.019073]$), a 4.19% reduction relative to the forced-promotion ablation. The raw one-sided randomization p-value is 1.0×10^{-5} and the Holm-adjusted value is 4.0×10^{-5}. Guarded DQL also reduces backlog AUC by 9.34% relative to blind promotion and by 1.41% relative to static placement. Its final backlog ratio is 0.35875, compared with 0.40936, 0.48584, and 0.36864, respectively. The throughput differences have the same direction: DQL delivers 43.02 million messages, 2.35 million more than the forced-promotion ablation and 5.91 million more than blind promotion.

Table S16. Adverse-probe ablation outcomes. Backlog AUC and final backlog are normalized by initial backlog; lower is better. Throughput is cumulative over 500 intervals. The 20k cohort is descriptive ($n_{\mathrm{ep}} = 5$).

Cohort	Policy	Backlog AUC	Final ratio	Throughput (M)	Mean pressure
1.2k inference	Frozen DQL	0.66046	0.35875	43.02	0.5873
1.2k inference	DQL + forced promotion	0.68938	0.40936	40.66	0.6100
1.2k inference	Blind promotion	0.72853	0.48584	37.11	0.5874
1.2k inference	Guarded heuristic	0.59104	0.25384	47.90	0.5941
1.2k inference	Static	0.66993	0.36864	42.56	0.5870
20k confirmation	Frozen DQL	0.95155	0.90316	97.05	0.7877
20k confirmation	DQL + forced promotion	0.95556	0.91076	91.16	0.7884
20k confirmation	Blind promotion	0.97323	0.94645	63.50	0.7875
20k confirmation	Guarded heuristic	0.94828	0.89658	102.15	0.7883
20k confirmation	Static	0.95452	0.90903	92.50	0.7875

The heuristic provides an important comparison. Its primary-cohort backlog AUC is 0.59104, so DQL is 11.75% higher (paired difference 0.069419, 95% CI $[0.066977, 0.071826]$). The action records explain the difference. Across 15,000 primary decision intervals, the heuristic executes 2,286 backlog-rebalancing actions, whereas DQL executes 175 backlog and 175 subscription-count moves and otherwise stays. In the 20k cohort, the heuristic uses all 625 interventions for backlog rebalancing. DQL splits 625 interventions almost evenly between 315 backlog and 310 subscription-count moves. The backlog-specific measure therefore favors the heuristic, which makes more backlog-focused moves. The ablation establishes the benefit of guarding probe promotion; the backlog-focused heuristic retains the stronger queue-drainage result.

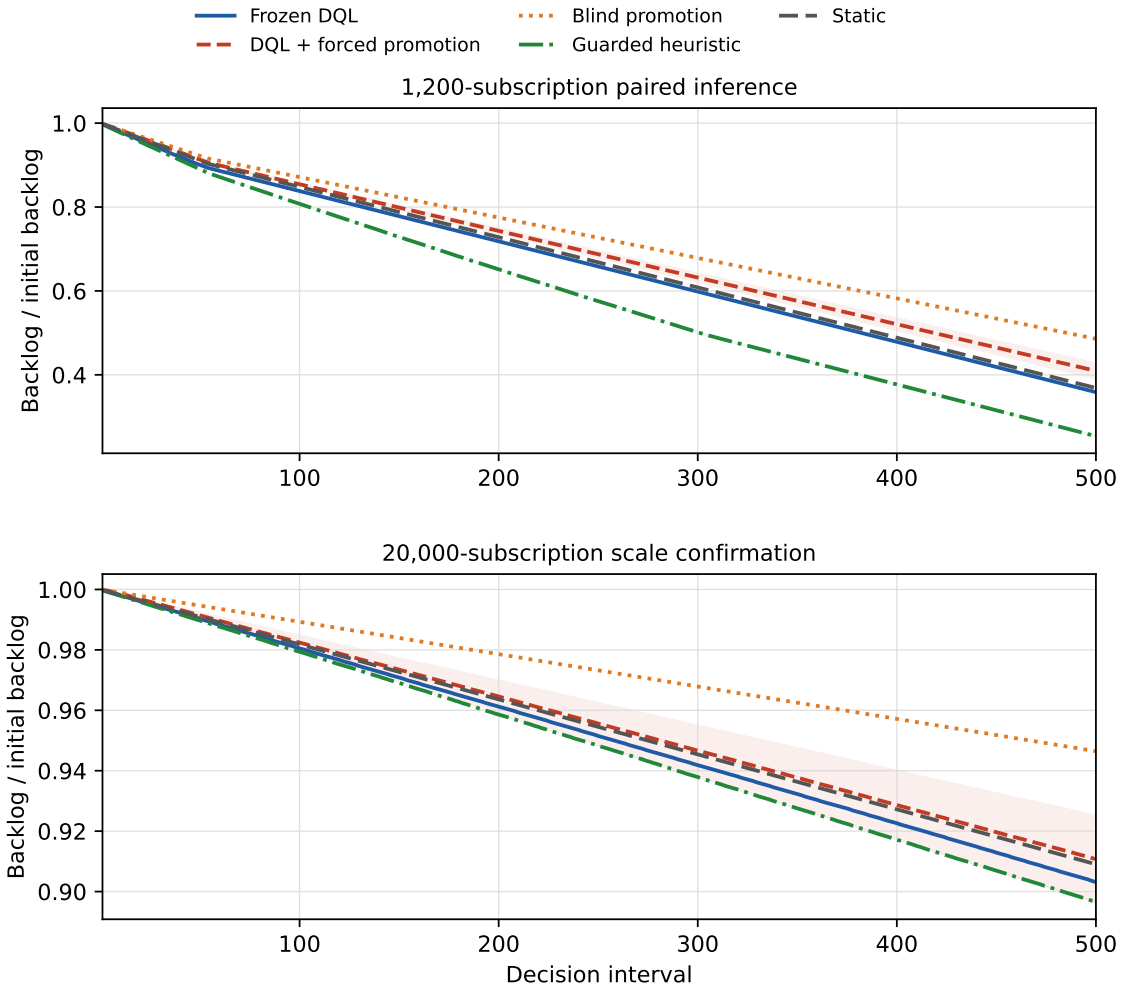


Fig. S19. Mean normalized backlog trajectories for the adverse-probe ablation. Shaded bands are descriptive 95% normal-approximation intervals across paired seeds. The upper panel contains the 30-seed inference cohort; the lower panel contains the five-seed 20,000-subscription scale confirmation. The forced-promotion policy resumes the same frozen DQL controller after the interval-zero override.

The 20,000-subscription cohort retains the requested four equal groups: 5,000 subscriptions at 100,000 queued messages, 5,000 at 50,000, 5,000 at 5,000, and 5,000 at zero, totaling 775 million initial messages. Guarded DQL reduces backlog AUC by 0.42% relative to forced promotion, 2.23% relative to blind promotion, and 0.31% relative to static placement, while remaining 0.34% above the heuristic. The forced-promotion paired interval lies below zero, but its exact one-sided $p = 0.0625$ and Holm-adjusted $p = 0.125$ reflect the five-seed sample. We consequently treat this cohort as scale confirmation, not as a second inferential test. The maximum normalized message-conservation residual over all 175 episodes is 1.031×10^{-14}; this verifies simulator accounting but does not establish loss-free production handoff.

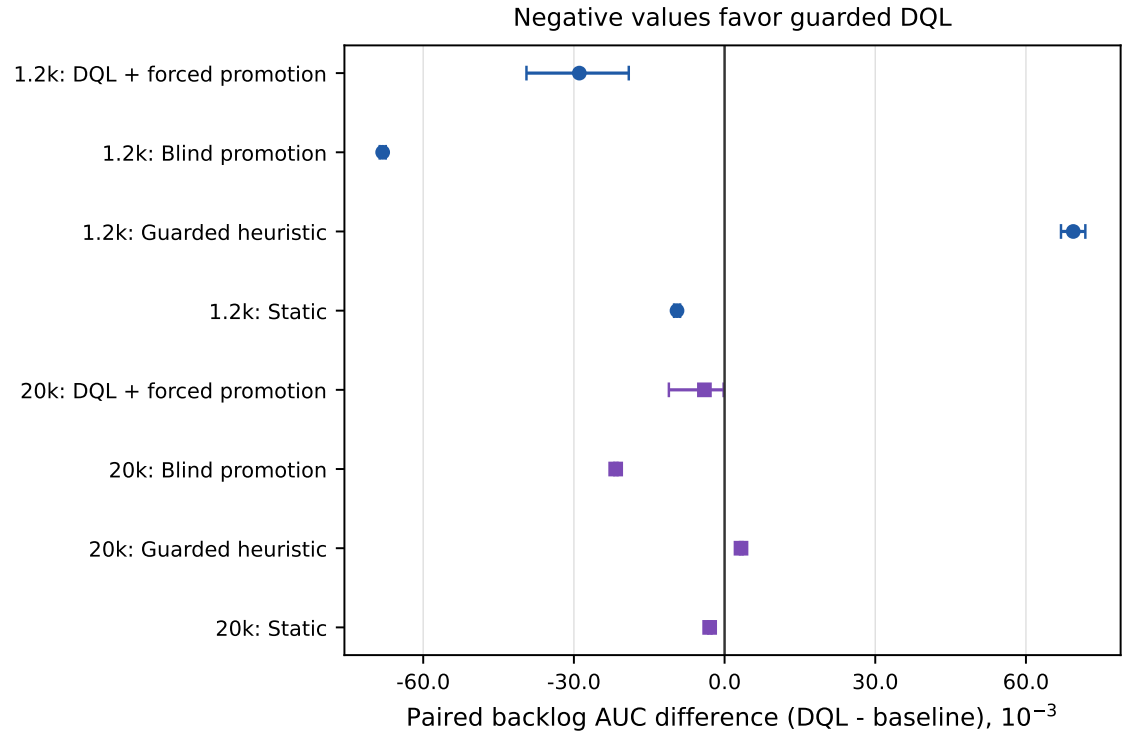


Fig. S20. Paired normalized backlog-AUC differences between guarded DQL and each baseline. Negative values favor DQL; whiskers are paired-bootstrap 95% confidence intervals. Squares denote the descriptive 20,000-subscription cohort.

K.8 Reproducibility and Study Scope

Reviewer artifact access. The accompanying supplementary archive, `reviewer-evidence.zip`, contains episode-level simulator CSVs, declared protocols, original aggregate analyses, the frozen 21–32–6 checkpoint, and identifier-free live summaries. Its README gives the file map and exact offline commands. A Python 3.9+ standard-library script verifies the checksum manifest, checks paired cohort coverage, recomputes policy means and selected effect sizes, and reconciles the loaded trial's reported subscription and ACK totals. A separate inference option evaluates the saved network without transport access or model updates. The archive supports reanalysis and checkpoint inspection; it does not expose corporate credentials, raw event payloads, private endpoint identifiers, or the proprietary service source. It is not a complete independent deployment rerun. Provenance records distinguish unchanged measurements from redacted local-path metadata.

The manuscript bundle contains the exact v3 aggregate CSV files, v3–v6 JSON protocols and summaries, the adverse-probe protocol and episode-level outputs, and the scripts that regenerate every figure in this section. The larger private evidence archive additionally records source-file SHA-256 hashes, frozen-checkpoint hashes, raw episode rows, and a recursive checksum manifest. The implementation test suite contains unit tests for state construction, action masks, replay, checkpoint persistence, migration state transitions, and fault guards, together with a Java STOMP fixture.

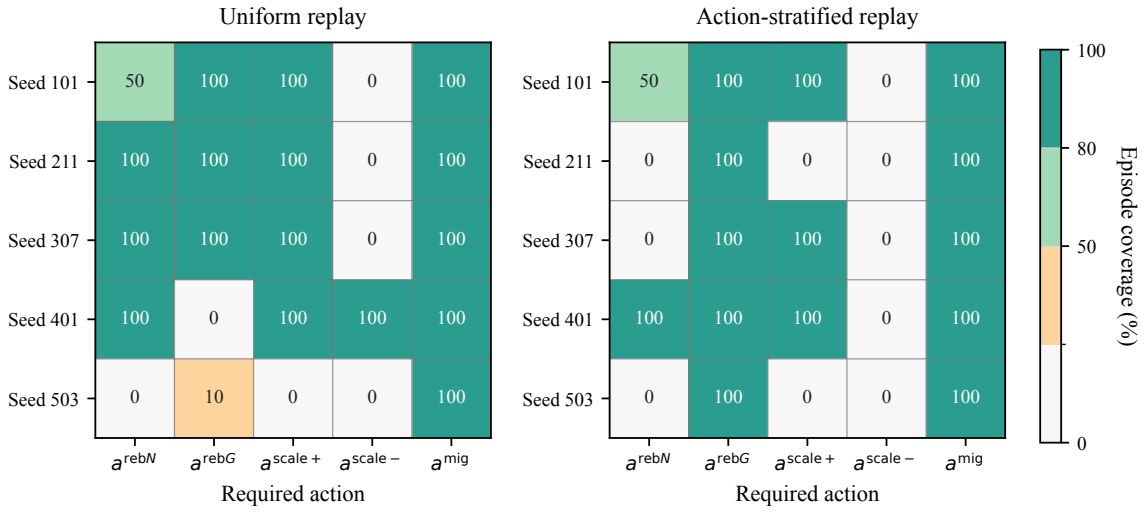


Fig. S21. Fixed-validation episode coverage for required actions in the two v4 development runs. A cell gives the percentage of admissible episodes in which the frozen candidate selected the indicated action; the predeclared pass threshold was 80% for every column in one row.

At the source revision used for this manuscript, all 267 tests pass, including the four adverse-probe evaluator tests, cap-restoring scale-up tests, and legacy-policy rejection tests.

The recovery study includes 20,000 subscriptions and four modeled pods; the earlier four-scenario holdout contains up to 1,600 subscriptions. These are simulator measurements with explicit workload and service assumptions. The v4–v6 all-action studies ended at their development coverage gates and are reported as diagnostics, separate from holdout performance. The Oracle Cloud cluster experiment in Section 6 of the main paper measures 20,000-subscription topology and ten live handoffs using existing queued events rather than the simulator's paired, controlled workloads. The backlog-recovery effect sizes therefore apply to the stated simulator. The Oracle Cloud measurements establish execution of the corresponding connection-replacement mechanism in Kubernetes. The additional loaded scale-out run in Section 6.4 of the main paper records three selective DQL handoffs and 8,000 acknowledged STOMP deliveries for 1,000 subscriptions. It uses a fixed client-processing delay and bounded publication, and supplies functional handoff evidence rather than a matched throughput effect size.

L Action-Contract Development Diagnostics

The v3 protocol validated the three actions central to the claimed rebalancing mechanism, but it did not require scale-up and scale-down. We therefore declared a stricter v4 gate before running a new study. Each of a^{rebN}, a^{rebG}, $a^{\text{scale+}}$, $a^{\text{scale-}}$, and a^{mig} had to be selected in at least 80% of admissible episodes in its own fixed validation scenario. Every candidate also had to pass four probe and telemetry fault guards. Five seeds were trained for 20,000 transitions with uniform replay and again with action-stratified replay, giving 100,000 exact training transitions in each development run.

All ten candidates passed the scripted scenario-feasibility checks and every fault-safety contract. No row in Figure S21 passes all five policy-coverage conditions. Uniform-replay seed 401 exercises scale-down but misses backlog rebalancing;

Table S17. V6 development-gate results. Each action and safety entry is the number of passing candidates out of five. TM denotes Target-Max bootstrap; DDQL denotes Double-DQL. An arm required at least four candidates that each passed all action and safety gates.

Arm	Learner/loss	Eligible	$a^{\mathrm{reb}N}$	$a^{\mathrm{reb}G}$	$a^{\mathrm{scale+}}$	$a^{\mathrm{scale-}}$	a^{mig}	Safety
A	TM/squared	0	0	1	0	5	5	5
B	DDQL/squared	0	2	2	0	5	5	5
C	TM/Huber	0	3	5	1	5	5	5
D	DDQL/Huber	0	4	5	0	5	5	5

the other uniform candidates miss scale-down. Every action-stratified candidate misses scale-down, and several also miss subscription rebalancing or scale-up. Action-stratified replay left gaps in frozen-policy action coverage. Under the declared stopping rule, both v4 studies ended at the development gate and retained their checkpoints for diagnosis. Their results concern action coverage rather than new holdout performance estimates.

V5 then changed only the scale-down admissibility rule and used fresh candidate and validation seeds. The historical target-metric checks passed, including a 49.6% connection-cost reduction for scale-down. All five frozen candidates passed every fault guard and selected $a^{\mathrm{reb}G}$, $a^{\mathrm{scale+}}$, and a^{mig} in all ten intended validation episodes. However, each selected $a^{\mathrm{reb}N}$ and $a^{\mathrm{scale-}}$ in zero of ten intended episodes. For the diagnostic candidate with the best reward, we evaluated each missed action for one step from the same state. Both improved reward, but its Q-values ranked *stay* higher. A later audit showed that the old $a^{\mathrm{scale+}}$ action reduced its declared maximum-subscription metric by 21.78% but lowered mean scalar reward by 0.04946 relative to static placement. The v5 diagnostics therefore distinguish missed one-step reward gains from a scale-up implementation that conflicted with its reward objective. The coverage gate was not met, and the study ended without a holdout evaluation.

V6 changed only the scale-up contract. The revised $a^{\mathrm{scale+}}$ opens one connection and performs bounded, receipt-gated $a^{\mathrm{reb}N}$ moves until the subscription target is restored or no eligible move remains. In the locked ten-seed preflight, the option improved mean interval reward by 0.04350 with a paired bootstrap 95% interval of [0.03559, 0.05114]. It reduced the mean maximum subscriptions per connection by 38.824, with interval [-38.824, -38.824]. Every episode restored the target using exactly 50 moves, without an executor no-op or scale-down reversal. This scripted preflight verifies the revised action contract in the simulator; learned selection is evaluated in the study below.

The locked v6 study then tested each combination of Target-Max or Double-DQL bootstrapping with squared or Huber temporal-difference loss. Each arm trained five fresh candidates for 20,000 transitions and evaluated them on ten fresh development-validation seeds. Table S17 reports the number of candidates passing each action gate. Every candidate passed all four safety scenarios, but no candidate passed all five action gates. The predeclared requirement of at least four fully eligible candidates was therefore unmet in every arm. No policy was selected and no holdout was defined or executed. Squared and Huber loss magnitudes are not ranked against each other because the two losses have different numerical scales.

The cap-restoring executor improves the fixed reward, while learned selection varies across candidates. The diagnostics distinguish the corrected action contract from policy coverage. They leave the individual effects of value estimation, visited states, replay sampling, and credit across several moves unresolved.

M Extended Interpretation

The complete interpretation from the reference manuscript is retained below. The main systems finding is that subscriber capacity and client access to that capacity are separate. Adding Pods supplies service capacity; moving existing subscription-bearing connections makes it reachable by persistent clients. The guarded-greedy and DQL policies both achieve this in the scale-out model. Their nearby throughput results support the shared mechanism rather than requiring a claim that learning beats a strong greedy rule in every workload.

Policy choice depends on the operational objective. Greedy control gives a direct, inspectable response to measured pressure. DQL can account for delayed costs and several competing objectives. A subscription move changes future backlog distribution; migration trades immediate handoff cost for later capacity. Both selectors operate above the same bounded, checked actions.

QEdgeProxy selects service instances using client QoS feedback [12], while Yahya et al. adapt offloading through decentralized bandits and server-reward adjustment [34]. The present work focuses on subscription-aware persistent messaging clients under partial observability and treats connection-level rebalancing as a first-class control action alongside endpoint selection. This distinction matters operationally: endpoint selection chooses the serving Pod, while connection-level rebalancing corrects subscription skew within the client.

The holdout results show how learning compares with the heuristic. Both policies can perform the same actions, so they can reach the same capacity limit in some scenarios. DQL's measured advantage is concentrated in subscription distribution, combined connection imbalance, and avoiding a harmful heuristic intervention during client slowdown. The heuristic retains an advantage in throughput and backlog-distribution metrics. These results identify the operating objectives favored by each policy. The component-level analysis reports these tradeoffs alongside the scalar reward; policy selection uses development validation before the holdout is evaluated.

The distinction between a model's quality and the utility of the complete system also appears in Casimiro et al.'s probabilistic framework for adapting ML components [4]. In our setting, a favorable Q-value is therefore interpreted alongside backlog, throughput, and handoff outcomes. Gheibi and Weyns introduce a lifelong-learning layer to respond to changing adaptation options [16]. Our experiments evaluate a fixed policy under the stated workload and deployment conditions. Lifelong adaptation is outside this study's scope.

The loaded Oracle Cloud scale-out trial adds direct evidence for selective execution (Section 6.4 of the main paper). Every handoff preserved three unselected active sessions, while the full cohort reached 8,000 ACK receipts. The measured drain waits also show why opening a replacement socket is only one part of migration: client processing and acknowledgement completion can dominate the handoff duration. These measurements validate the learned decision-to-execution path under the declared load and processing limit.

The action-contract studies provide a further design lesson: a useful executor and a policy that reliably selects it are separate requirements. The cap-restoring scale-up preflight improved reward and maximum subscription count, but the expanded learner study did not pass every coverage gate. Appendix L retains these results. They explain why the live checkpoint uses a compatible, restricted rebalancing profile.

M.1 Conclusion

We presented a probe-guided, client-side framework for rebalancing persistent subscription connections without requiring cluster-wide client visibility. The mechanism combines per-subscription queues, active-connection telemetry, and a load-balancer-assigned probe. Greedy rules or DQL select bounded actions, and a shared executor carries out

receipt-gated handoff. The technical contribution is the link between partial observations, endpoint-preserving probe promotion, and selective subscription reassociation.

The primary experiment demonstrates recovery after a one-to-four pod capacity increase with 20,000 subscriptions and 775 million queued events. Guarded-greedy migration and frozen DQL deliver 221,260 and 219,762 events/s, respectively, compared with 58,000 events/s for unchanged placement. Frozen DQL uses all four pods, delivers 3.79 times the mean throughput, and leaves 27.52% less backlog after 20 minutes than retaining the ten original connection assignments. Its 12.30% reduction in backlog area satisfies the registered paired statistical criterion. The adverse-probe ablation complements this result: evaluating the candidate before promotion reduces backlog area by 4.19% relative to forcing one harmful migration.

These findings identify the operational benefit of the mechanism: scaled-out subscriber capacity becomes useful to existing subscriptions through client-directed connection reassociation, while combined subscription and backlog pressure determines whether a sampled target is suitable. The supporting studies characterize policy tradeoffs and action coverage. The mixed-workload holdout favors DQL in composite reward and subscription balance, while the heuristic favors throughput and backlog balance. These results support two evaluated policy options rather than universal DQL superiority or an untested DQL-tuned scripted policy. The isolated Oracle Cloud cluster qualification also demonstrates a 60% reduction in source subscriptions through three full-connection migrations, with all 2,000 subscriptions retained and 73,639 observed deliveries reconciled at closure. Its startup measurements explicitly capture activation while deliveries flow. A further loaded one-to-four-Pod trial retained all 1,000 subscriptions and four active connections through three DQL-selected handoffs. Each action preserved the other three sessions. The final client distribution was 500, 250, 250 and zero across the four available Pods, with 8,000 observed deliveries acknowledged and no unresolved client deliveries at closure. A separate Oracle Cloud preload pair measured twelve native TTL interruptions and 89.44% ACK-receipt completion, versus no interruptions and 100% completion in the TTL-disabled DQL arm (Section I.5). That arm remained at guard-held *stay*: a 200-second minimum telemetry sweep exceeded the 180-second freshness deadline. The result measures expiry-related receipt completion under the telemetry constraint. The comparative throughput and backlog-recovery gains are simulator measurements under the stated service assumptions. The Oracle Cloud experiments establish live handoff execution, subscription retention, and observed delivery behavior. Together, these studies describe the mechanism's measured benefits and the operating conditions under which they were obtained.

References

[1] Abeer Abdel Khaleq and Ilkyeun Ra. 2021. Intelligent Autoscaling of Microservices in the Cloud for Real-Time Applications. *IEEE Access* 9 (2021), 35464–35476. doi:10.1109/ACCESS.2021.3061890

[2] Ali Belgacem, Kadda Beghdad-Bey, Hassina Nacer, and Sofiane Bouznad. 2020. Efficient dynamic resource allocation method for cloud computing environment. *Cluster Computing* 23, 4 (2020), 2871–2889. doi:10.1007/s10586-020-03053-x

[3] Matteo Biagiola and Paolo Tonella. 2022. Testing the Plasticity of Reinforcement Learning-based Systems. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022), 1–46. doi:10.1145/3511701

[4] Maria Casimiro, Diogo Soares, David Garlan, Luís Rodrigues, and Paolo Romano. 2024. Self-adapting Machine Learning-based Systems via a Probabilistic Model Checking Framework. *ACM Transactions on Autonomous and Adaptive Systems* 19, 3 (2024), 1–30. doi:10.1145/3648682

[5] Luca Caviglione, Mauro Gaggero, Massimo Paolucci, and Roberto Ronco. 2021. Deep reinforcement learning for multi-objective placement of virtual machines in cloud datacenters. *Soft Computing* 25, 19 (2021), 12569–12588. doi:10.1007/s00500-020-05462-x

[6] Genxin Chen, Jin Qi, Ying Sun, Xiaoxuan Hu, Zhenjiang Dong, and Yanfei Sun. 2023. A collaborative scheduling method for cloud computing heterogeneous workflows based on deep reinforcement learning. *Future Generation Computer Systems* 141 (2023), 284–297. doi:10.1016/j.future.2022.11.032

[7] Xing Chen, Lijian Yang, Zheyi Chen, Geyong Min, Xianghan Zheng, and Chunming Rong. 2023. Resource allocation with workload-time windows for cloud-based software services: A deep reinforcement learning approach. *IEEE Transactions on Cloud Computing* 11, 2 (2023), 1871–1885. doi:10.1109/TCC.2022.3169157

[8] Xing Chen, Fangning Zhu, Zheyi Chen, Geyong Min, Xianghan Zheng, and Chunming Rong. 2022. Resource allocation for cloud-based software services using prediction-enabled feedback control with reinforcement learning. *IEEE Transactions on Cloud Computing* 10, 2 (2022), 1117–1129. doi:10.1109/TCC.2020.2992537

[9] Zheyi Chen, Jia Hu, Geyong Min, Chunbo Luo, and Tarek El-Ghazawi. 2022. Adaptive and efficient resource allocation in cloud datacenters using actor-critic deep reinforcement learning. *IEEE Transactions on Parallel and Distributed Systems* 33, 8 (2022), 1911–1923. doi:10.1109/TPDS.2021.3132422

[10] Zhen Chen, Lin Zhang, Xiaohan Wang, and Kunyu Wang. 2023. Cloud-edge collaboration task scheduling in cloud manufacturing: An attention-based deep reinforcement learning approach. *Computers & Industrial Engineering* 177 (2023), 109053. doi:10.1016/j.cie.2023.109053

[11] Long Cheng, Yue Wang, Feng Cheng, Cheng Liu, Zhiming Zhao, and Ying Wang. 2024. A deep reinforcement learning-based preemptive approach for cost-aware cloud job scheduling. *IEEE Transactions on Sustainable Computing* 9, 3 (2024), 422–432. doi:10.1109/TSUSC.2023.3303898

[12] Ivan Čilić, Ivana Podnar Žarko, Pantelis Frangoudis, and Schahram Dustdar. 2026. QoS-Aware Load Balancing in the Computing Continuum via Multi-Player Bandits. arXiv preprint arXiv:2512.18915v2. https://arxiv.org/abs/2512.18915v2

[13] Tongke Cui, Ruopeng Yang, Chao Fang, and Shui Yu. 2023. Deep Reinforcement Learning-Based Resource Allocation for Content Distribution in IoT-Edge-Cloud Computing Environments. *Symmetry* 15, 1 (2023), 217. doi:10.3390/sym15010217

[14] Ding Ding, Xiaocong Fan, Yihuan Zhao, Kaixuan Kang, Qian Yin, and Jing Zeng. 2020. Q-learning based dynamic task scheduling for energy-efficient cloud computing. *Future Generation Computer Systems* 108 (2020), 361–371. doi:10.1016/j.future.2020.02.018

[15] Tingting Dong, Fei Xue, Chuangbai Xiao, and Juntao Li. 2020. Task scheduling based on deep reinforcement learning in a cloud manufacturing environment. *Concurrency and Computation: Practice and Experience* 32, 11, Article e5654 (2020). doi:10.1002/cpe.5654

[16] Omid Gheibi and Danny Weyns. 2024. Dealing with Drift of Adaptation Spaces in Learning-based Self-Adaptive Systems Using Lifelong Self-Adaptation. *ACM Transactions on Autonomous and Adaptive Systems* 19, 1 (2024), 1–57. doi:10.1145/3636428

[17] Diego Hortelano, Ignacio de Miguel, Ramón J. Durán Barroso, Juan Carlos Aguado, Noemí Merayo, Lidia Ruiz, Adrian Asensio, Xavi Masip-Bruin, Patricia Fernández, Rubén M. Lorenzo, and Evaristo J. Abril. 2023. A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems. *Journal of Network and Computer Applications* 216 (2023), 103669. doi:10.1016/j.jnca.2023.103669

[18] Jiaxin Hou, Yingbo Wu, Junpeng Cai, and Zhiwen Zhou. 2023. QoE-guaranteed distributed offloading decision via partially observable deep reinforcement learning for edge-enabled internet of things. *Neural Computing and Applications* 35, 29 (2023), 21603–21619. doi:10.1007/s00521-023-08905-2

[19] Liang Huang, Suzhi Bi, and Ying-Jun Angela Zhang. 2020. Deep Reinforcement Learning for Online Computation Offloading in Wireless Powered Mobile-Edge Computing Networks. *IEEE Transactions on Mobile Computing* 19, 11 (2020), 2581–2593. doi:10.1109/TMC.2019.2928811

[20] Sundas Iftikhar, Sukhpal Singh Gill, Chenghao Song, Minxian Xu, Mohammad Sadegh Aslanpour, Adel N. Toosi, Junhui Du, Huaming Wu, Shreya Ghosh, Deepraj Chowdhury, Muhammed Golec, Mohit Kumar, Ahmed M. Abdelmoniem, Felix Cuadrado, Blesson Varghese, Omer Rana, Schahram Dustdar, and Steve Uhlig. 2023. AI-based fog and edge computing: A systematic review, taxonomy and future directions. *Internet of Things* 21 (2023), 100674. doi:10.1016/j.iot.2022.100674

[21] Shashikant Ilager, Jakob Fahringer, Alessandro Tundo, and Ivona Brandić. 2026. A Decentralized and Self-Adaptive Approach for Monitoring Volatile Edge Environments. *ACM Transactions on Autonomous and Adaptive Systems* 21, 2 (2026), 1–30. doi:10.1145/3736726

[22] Amanda Jayanetti, Saman Halgamuge, and Rajkumar Buyya. 2022. Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge-cloud computing environments. *Future Generation Computer Systems* 137 (2022), 14–30. doi:10.1016/j.future.2022.06.012

[23] Tahseen Khan, Wenhong Tian, Guangyao Zhou, Shashikant Ilager, Mingming Gong, and Rajkumar Buyya. 2022. Machine learning (ML)-centric resource management in cloud computing: A review and future directions. *Journal of Network and Computer Applications* 204 (2022), 103405. doi:10.1016/j.jnca.2022.103405

[24] Hongjian Li, Liang Lu, Wenhu Shi, Gangfan Tan, and Hao Luo. 2023. Energy-aware scheduling for Spark job based on deep reinforcement learning in cloud. *Computing* 105, 8 (2023), 1717–1743. doi:10.1007/s00607-023-01171-z

[25] Zhaoze Liu, Yajuan Sun, Cong Liu, Zhiming Zhao, and Long Cheng. 2026. Data-intensive task scheduling in geo-distributed cloud computing: a deep reinforcement learning approach. *Journal of Cloud Computing* 15, 1, Article 79 (2026), 15 pages. doi:10.1186/s13677-026-00894-6

[26] Giorgos Mitsis, Eirini Eleni Tsiropoulou, and Symeon Papavassiliou. 2022. Price and Risk Awareness for Data Offloading Decision-Making in Edge Computing Systems. *IEEE Systems Journal* 16, 4 (2022), 6546–6557. doi:10.1109/JSYST.2022.3188997

[27] Gorka Nieto, Idoia de la Iglesia, Unai Lopez-Novoa, and Cristina Perfecto. 2024. Deep Reinforcement Learning techniques for dynamic task offloading in the 5G edge-cloud continuum. *Journal of Cloud Computing* 13, 1, Article 94 (2024), 24 pages. doi:10.1186/s13677-024-00658-0

[28] Meini Pan, Zhihua Li, and Junhao Qian. 2023. Energy-efficient multiuser and multitask computation offloading optimization method. *Intelligent and Converged Networks* 4, 1 (2023), 76–92. doi:10.23919/ICN.2023.0007

[29] Andrew Patterson, Samuel Neumann, Martha White, and Adam White. 2024. Empirical Design in Reinforcement Learning. *Journal of Machine Learning Research* 25, 318 (2024), 1–63. https://jmlr.org/papers/v25/23-0183.html

[30] Yousef Sanjalawe, Salam Al-E'mari, Salam Fraihat, and Sharif Makhadmeh. 2025. AI-driven job scheduling in cloud computing: A comprehensive review. *Artificial Intelligence Review* 58, 7, Article 197 (2025), 113 pages. doi:10.1007/s10462-025-11208-8

[31] José Santos, Mattia Zaccarini, Filippo Poltronieri, Mauro Tortonesi, Cesare Stefanelli, Nicola Di Cicco, and Filip De Turck. 2025. HephaestusForge: Optimal microservice deployment across the Compute Continuum via Reinforcement Learning. *Future Generation Computer Systems* 166 (2025), 107680. doi:10.1016/j.future.2024.107680

[32] Wenting Wei, Huaxi Gu, Kun Wang, Jianjia Li, Xuan Zhang, and Ning Wang. 2023. Multi-dimensional Resource Allocation in Distributed Data Centers using Deep Reinforcement Learning. *IEEE Transactions on Network and Service Management* 20, 2 (2023), 1817–1829. doi:10.1109/TNSM.2022.3213575

[33] Jianqiao Xu, Zhuohan Xu, and Bing Shi. 2022. Deep Reinforcement Learning Based Resource Allocation Strategy in Cloud-Edge Computing System. *Frontiers in Bioengineering and Biotechnology* 10 (2022), 908056. doi:10.3389/fbioe.2022.908056

[34] Mariam Yahya, Alexander Conzelmann, and Setareh Maghsudi. 2024. Decentralized Task Offloading and Load-Balancing for Mobile Edge Computing in Dense Networks. *IEEE Communications Letters* 28, 8 (2024), 1954–1958. doi:10.1109/LCOMM.2024.3416833

[35] Guangyao Zhou, Wenhong Tian, Rajkumar Buyya, Ruini Xue, and Liang Song. 2024. Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions. *Artificial Intelligence Review* 57, 5, Article 124 (2024), 42 pages. doi:10.1007/s10462-024-10756-9

[36] Guangyao Zhou, Ruiming Wen, Wenhong Tian, and Rajkumar Buyya. 2022. Deep reinforcement learning-based algorithms selectors for the resource scheduling in hierarchical Cloud computing. *Journal of Network and Computer Applications* 208 (2022), 103520. doi:10.1016/j.jnca.2022.103520